

Large Language Models

Aula 16 - Machine Learning II

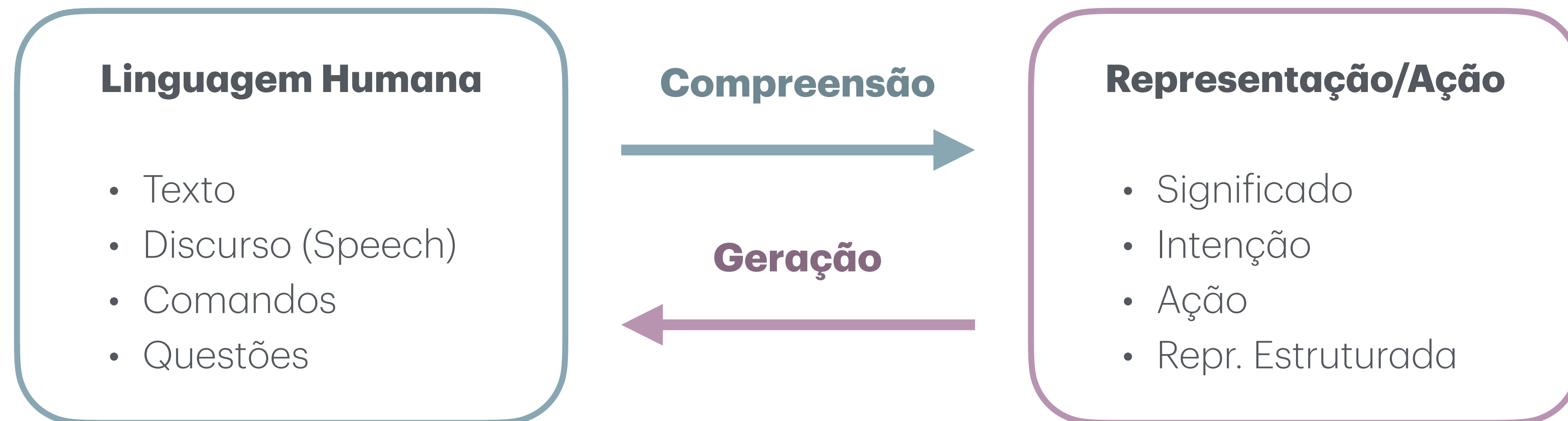
Daniel Yukimura, Francisco Ganacim
IMPA

1 de junho de 2026

Modelos de linguagem

Processamento de Linguagem Natural (NLP) possui dois objetivos principais:

- Compreender Linguagem
- Gerar Linguagem



Modelos de linguagem

Um **modelo de linguagem** é uma distribuição de probabilidade p no espaço de possíveis sequências finitas de “palavras” $\mathcal{V}^*$ (string).

$$\mathcal{V}^* = \bigcup_{n=1}^{\infty} \mathcal{V}^n$$

$\mathcal{V}$ é o vocabulário — conjunto de possíveis tokens.

Modelos de linguagem

Dada uma string $\mathbf{x} = x_1x_2\dots x_n \in \mathcal{V}^n$ podemos decompor a probabilidade p em fatores das probabilidades de obter uma palavra dada as anteriores

$$p(\mathbf{x}) = \prod_{k=1}^n p\left(x_k \mid \mathbf{x}_{1:k-1}\right)$$

onde $\mathbf{x}_{1:k-1} = x_1x_2\dots x_{k-1}$ (também escrevemos como $\mathbf{x}_{<k}$).

Modelos de linguagem

Um **modelo de linguagem autoregressivo** é um modelo de linguagem definido a partir das probabilidades condicionais $p(x_t | \mathbf{x}_{<t})$, onde $x_t \in \mathcal{V}$ e $\mathbf{x}_{<t} \in \mathcal{V}^*$.

Essa abordagem traduz o problema de **prever a próxima palavra** em um modelo de linguagem generativo.

No contexto de LLMs, modelamos as condicionais $p_\theta(\cdot | \mathbf{x})$ usando uma rede neural.

Modelos de linguagem

Seja $\mathcal{D} \subset \mathcal{V}^*$ um conjunto de treino. O log-likelihood de cada sentença $\mathbf{x} = x_1 \dots x_k$ em $\mathcal{D}$ pode ser calculada como

$$\mathcal{L}_\theta(\mathbf{x}) = \sum_{i=1}^k \log p_\theta \left(x_i \mid \mathbf{x}_{1:i-1} \right)$$

Então, um modelo de linguagem pode ser treinado para maximizar a verossimilhança

$$\hat{\theta} = \arg \max_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \mathcal{L}_\theta(\mathbf{x})$$

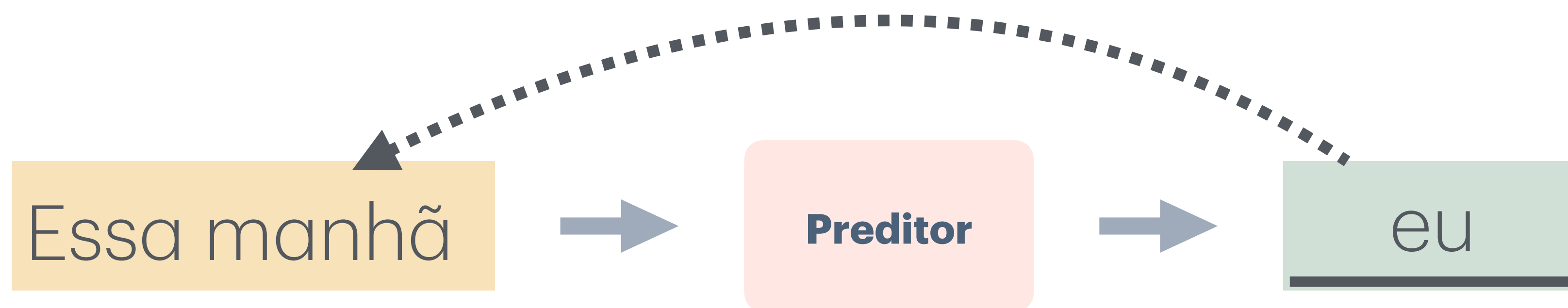
Previendo a próxima palavra

Essa manhã eu levei meu cachorro para passar

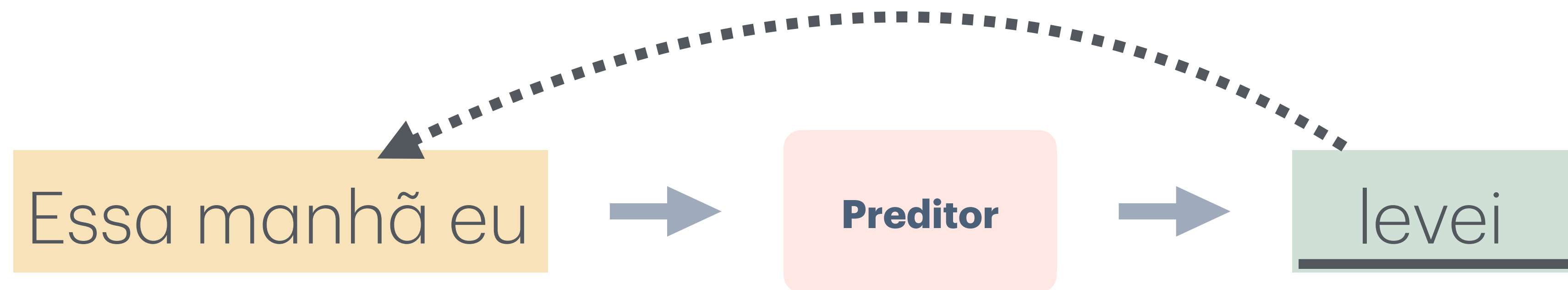
contexto

previsão

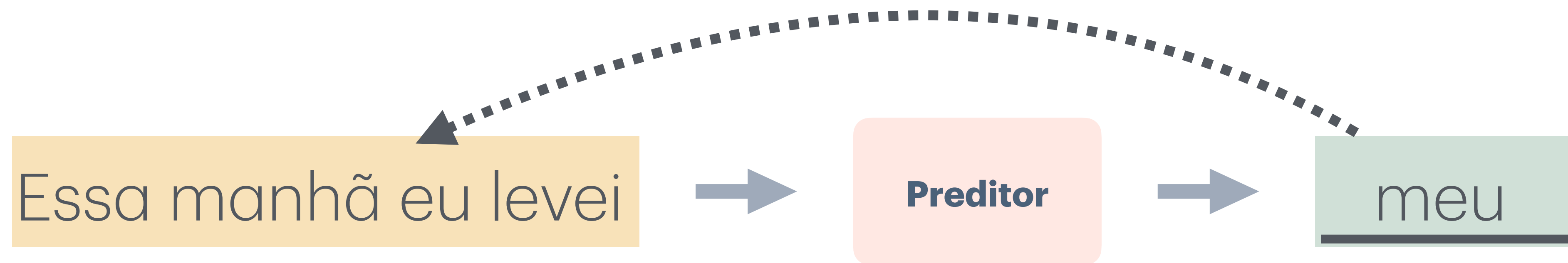
Previendo a próxima palavra



Previendo a próxima palavra

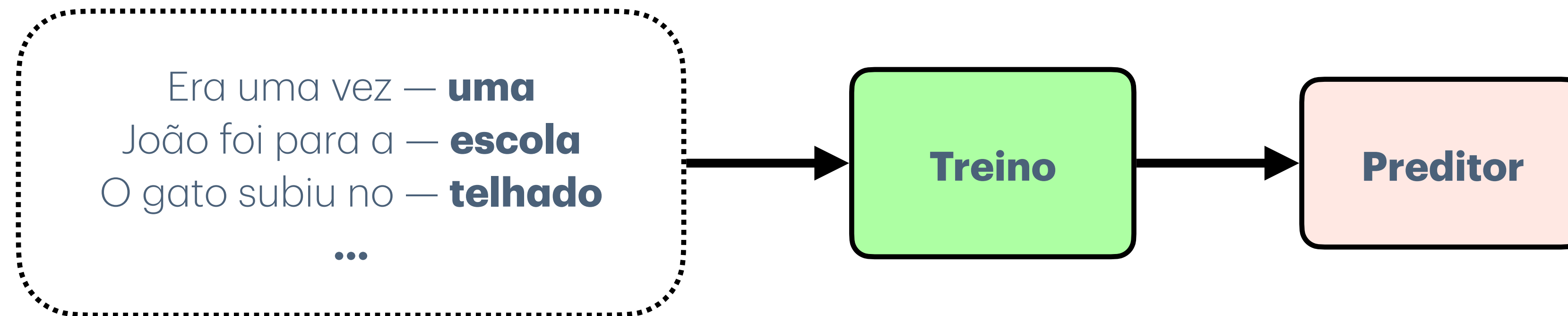


Previendo a próxima palavra

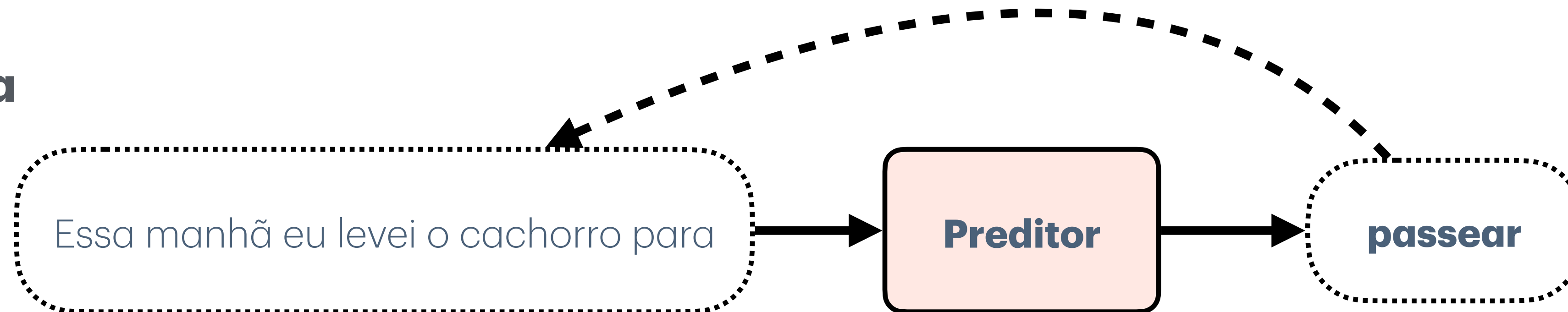


Previendo a próxima palavra

Treinamento



Inferência



Modelos de linguagem

Exemplo: Modelo n-grama

A ideia é assumir uma propriedade de Markov — x_t depende apenas dos últimos $n - 1$ tokens do contexto.

$$p(x_t \mid \mathbf{x}_{<t}) = p(x_t \mid \mathbf{x}_{t-n+1:t-1})$$
$$= \frac{C(\mathbf{x}_{t-n+1:n-1}x_n)}{C(\mathbf{x}_{t-n+1:n-1})}$$

frequência relativa

Número de vezes que
 $\mathbf{x}_{t-n+1:n-1}x_n$ **aparece no**
dataset.

Modelos de linguagem

Exemplo: Modelo n-grama

$$p(\text{sala} \mid \text{o gato dormia no sofá da}) = \frac{C(\text{o gato dormia no sofá da, sala})}{C(\text{o gato dormia no sofá da})}$$

No caso de $n=3$

$$p(\text{sala} \mid \text{o gato dormia no sofá da}) = p(\text{sala} \mid \text{sofá da}) = \frac{C(\text{sofá da, sala})}{C(\text{sofá da})}$$

Modelos de linguagem

Exemplo: Modelo 3-grama

$$p(\text{o gato dormia no sofá da sala}) = p(\text{o}) p(\text{gato} \mid \text{o}) p(\text{dormia} \mid \text{o gato})$$

$$p(\text{no} \mid \text{gato dormia}) p(\text{sofá} \mid \text{dormia no})$$

$$p(\text{da} \mid \text{no sofá}) p(\text{sala} \mid \text{sofá da})$$

Modelos de linguagem

Exemplo: Modelo 3-grama

$$\begin{aligned} p(\text{o gato dormia no sofá da sala}) &= p(\text{o} \mid [\text{BOS}]) p(\text{gato} \mid [\text{BOS}] \text{o}) p(\text{dormia} \mid \text{o gato}) \\ &\quad p(\text{no} \mid \text{gato dormia}) p(\text{sofá} \mid \text{dormia no}) \\ &\quad p(\text{da} \mid \text{no sofá}) p(\text{sala} \mid \text{sofá da}) p([\text{EOS}] \mid \text{da sala}) \end{aligned}$$

É importante considerar o começo e fim das frases!

Modelos de linguagem

Observação: A probabilidade induzida pelo modelo autoregressivo precisa satisfazer

$$p(\mathcal{V}^*) = 1$$

ou seja, com probabilidade **1 a sentença gerada precisa terminar** (prever [EOS]).

- Sabemos que para os Transformers usados em LLMs isso é verdade - Cotterell et al. 2023

Modelos de linguagem

Large Language Models in Machine Translation

Thorsten Brants Ashok C. Popat Peng Xu Franz J. Och Jeffrey Dean

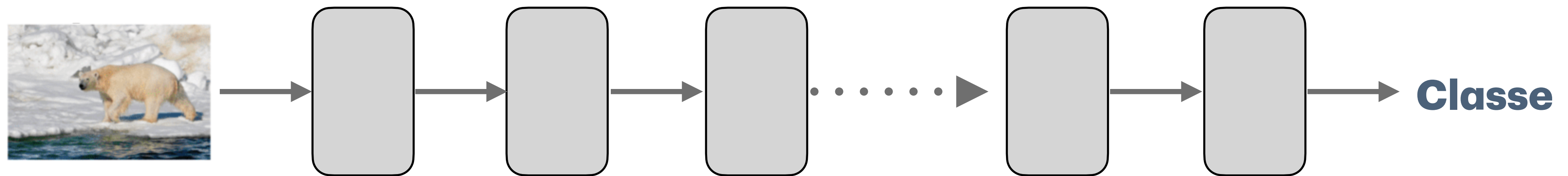
Google, Inc.
1600 Amphitheatre Parkway
Mountain View, CA 94303, USA
`{brants,popat,xp,och,jeff}@google.com`

Abstract

This paper reports on the benefits of large-scale statistical language modeling in machine translation. A distributed infrastructure is proposed which we use to train on up to 2 trillion tokens, resulting in language models having up to 300 billion n -grams. It is capable of providing smoothed probabilities for fast, single-pass decoding. We introduce a new smoothing method, dubbed *Stupid Backoff*, that is inexpensive to train on large data sets and approaches the quality of Kneser-Ney Smoothing as the amount of training data increases.

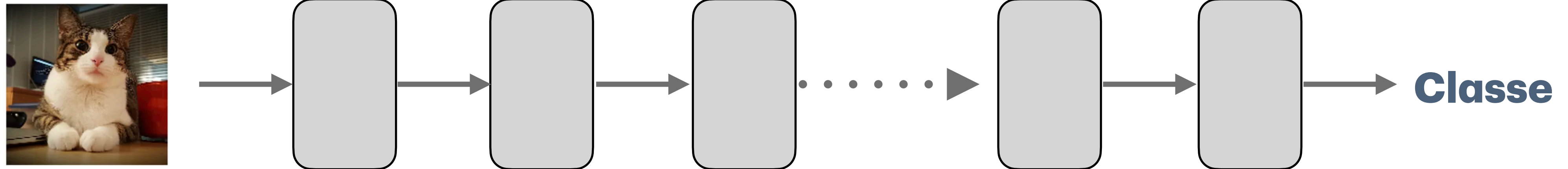
Pré-Treinamento

Pré-treino



IMAGENET

Fine-tuning



Oxford-IIIT-Pet

Pré-Treinamento

- **Word2Vec** e **GLOVE** são exemplos de **pré-treino não-supervisionado** para modelos de linguagem.

Distributed Representations of Words and Phrases and their Compositionality

Tomas Mikolov
Google Inc.
Mountain View
mikolov@google.com

Ilya Sutskever
Google Inc.
Mountain View
ilyasu@google.com

Kai Chen
Google Inc.
Mountain View
kai@google.com

Greg Corrado
Google Inc.
Mountain View
gcorrado@google.com

Jeffrey Dean
Google Inc.
Mountain View
jeff@google.com

GloVe: Global Vectors for Word Representation

Jeffrey Pennington, Richard Socher, Christopher D. Manning
Computer Science Department, Stanford University, Stanford, CA 94305
jpennin@stanford.edu, richard@socher.org, manning@stanford.edu

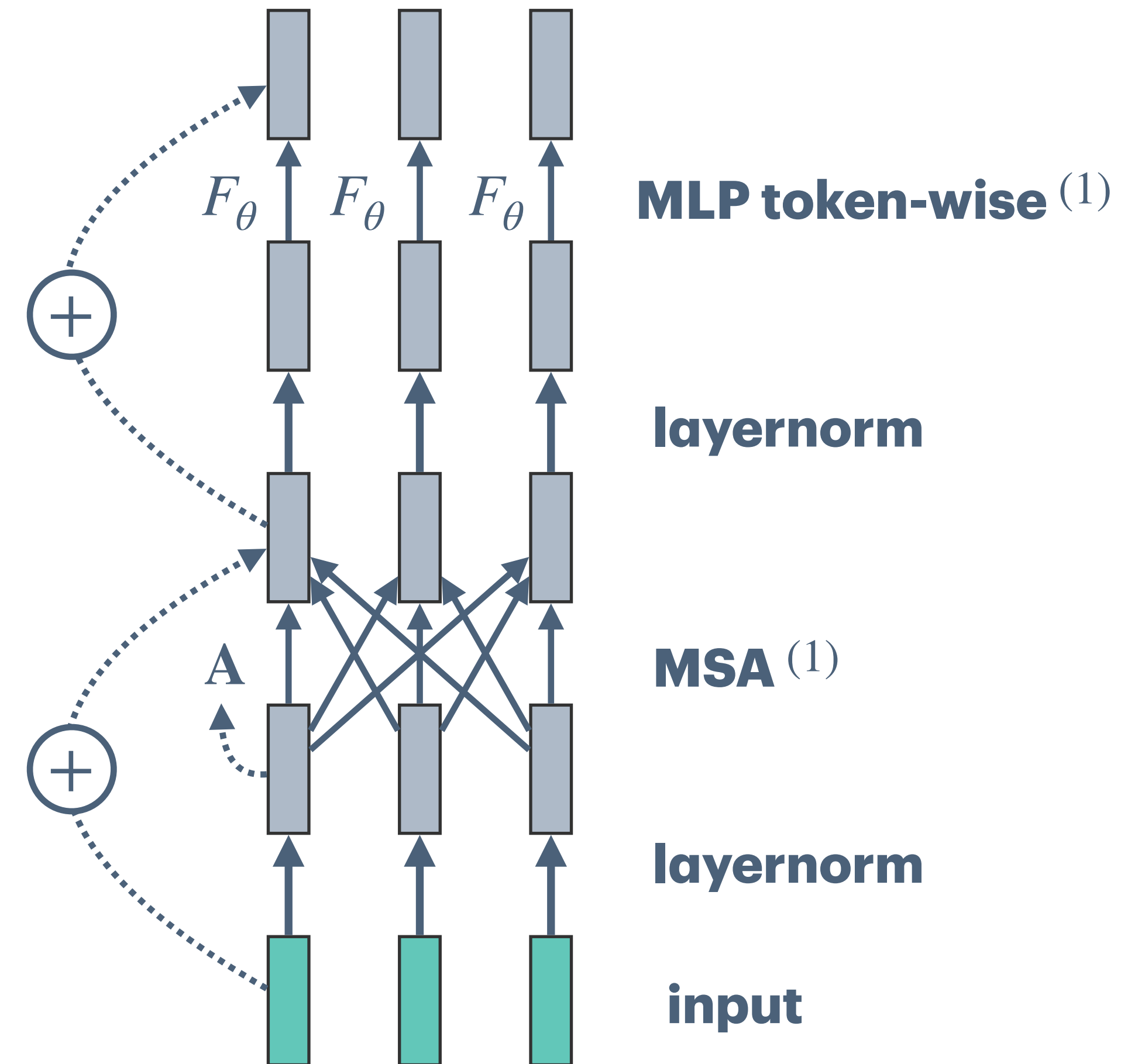
Distributed Representations of Words and Phrases and their Compositionality — Mikolov et al. 2013

GloVe: Global Vectors for Word Representation — Pennington et al. 2014

Transformers

Componentes de um Transformer:

- Tokenizador
- Positional Embedding
- Atenção
- FFN (MLP) + Ativação
- Normalização (quando e como)
- ...

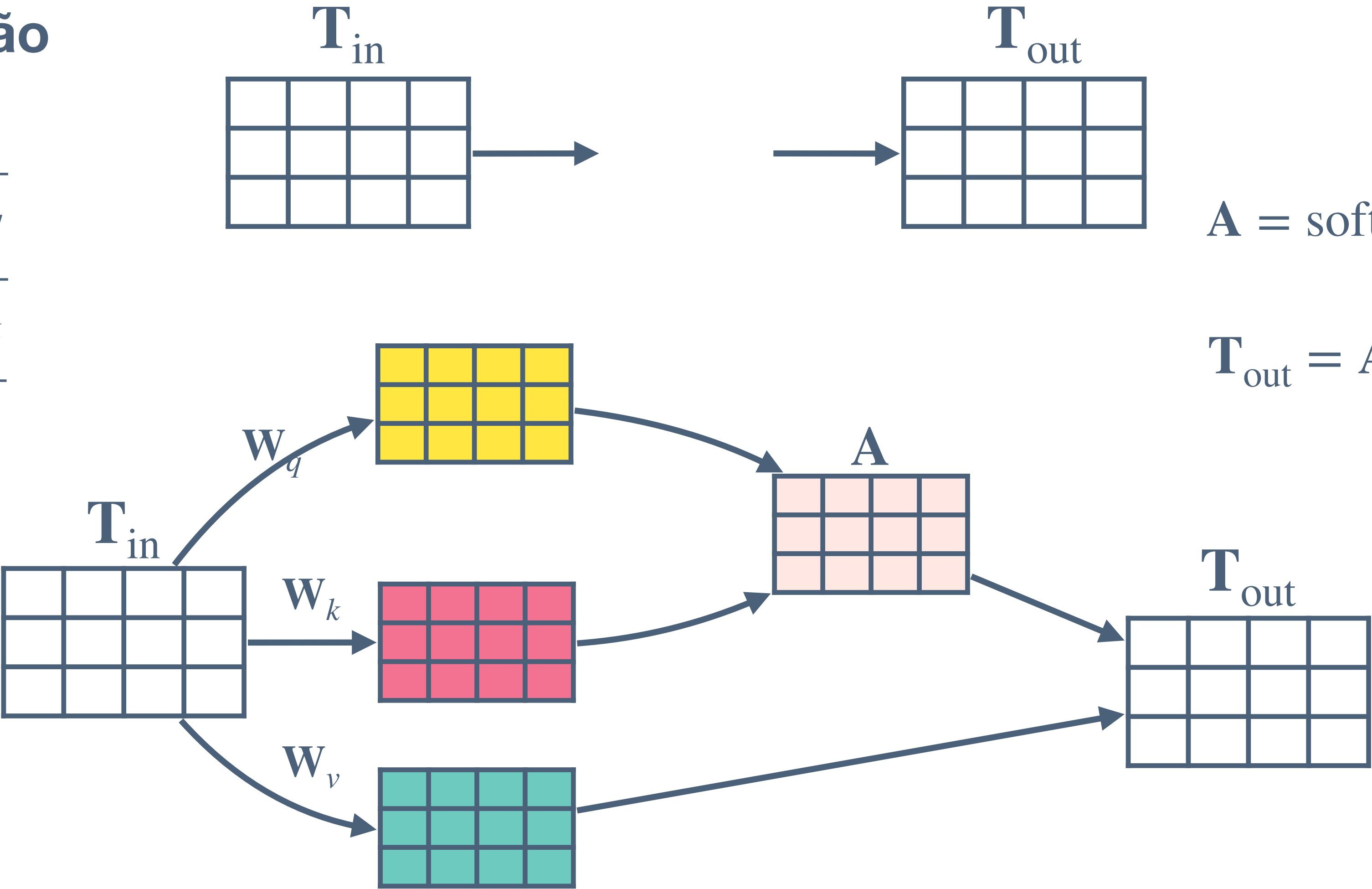


Auto-Atenção

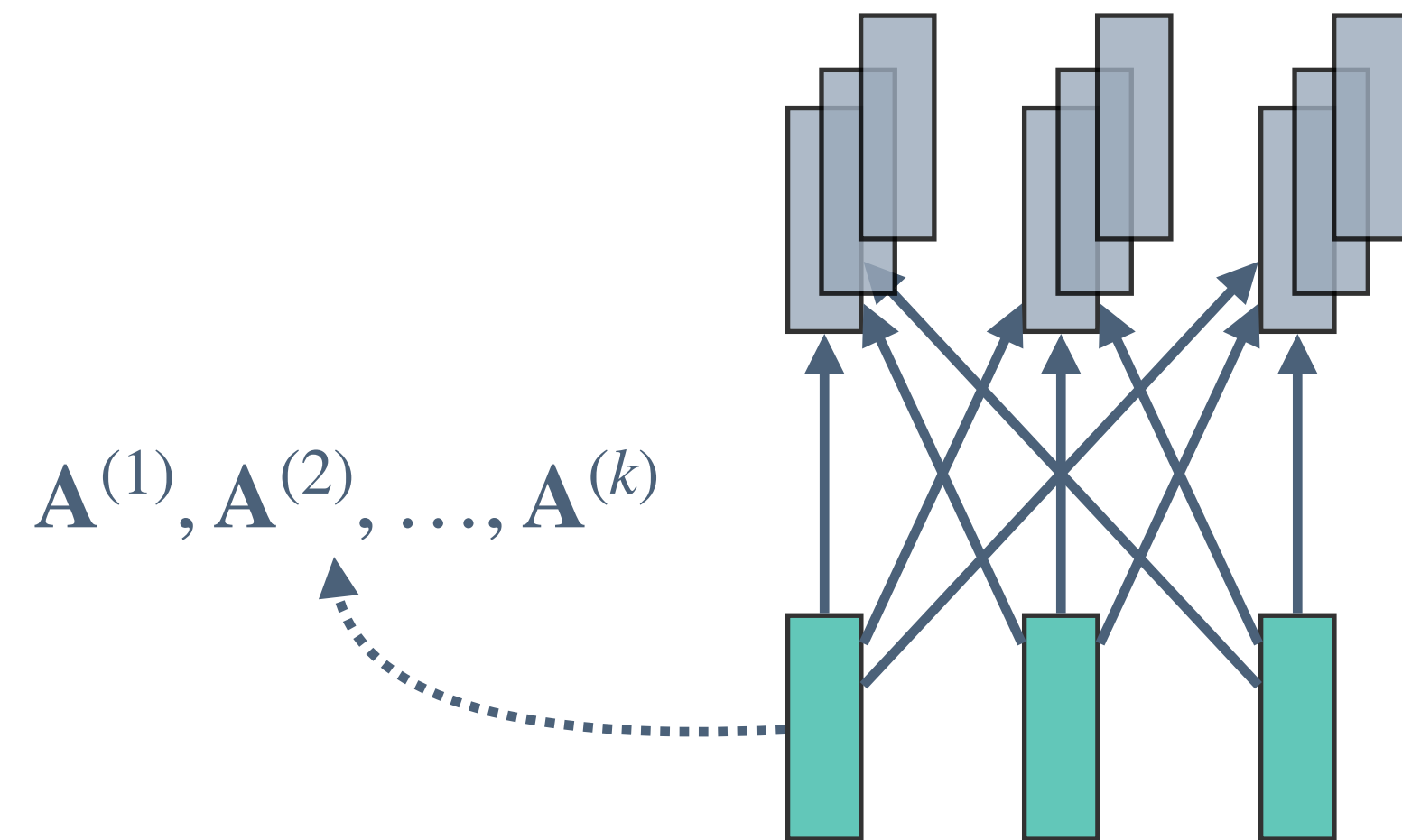
$$\mathbf{Q}_{\text{in}} = \mathbf{T}_{\text{in}} \mathbf{W}_q^\top$$

$$\mathbf{K}_{\text{in}} = \mathbf{T}_{\text{in}} \mathbf{W}_k^\top$$

$$\mathbf{V}_{\text{in}} = \mathbf{T}_{\text{in}} \mathbf{W}_v^\top$$



Auto-atenção com múltiplas cabeças (MSA)

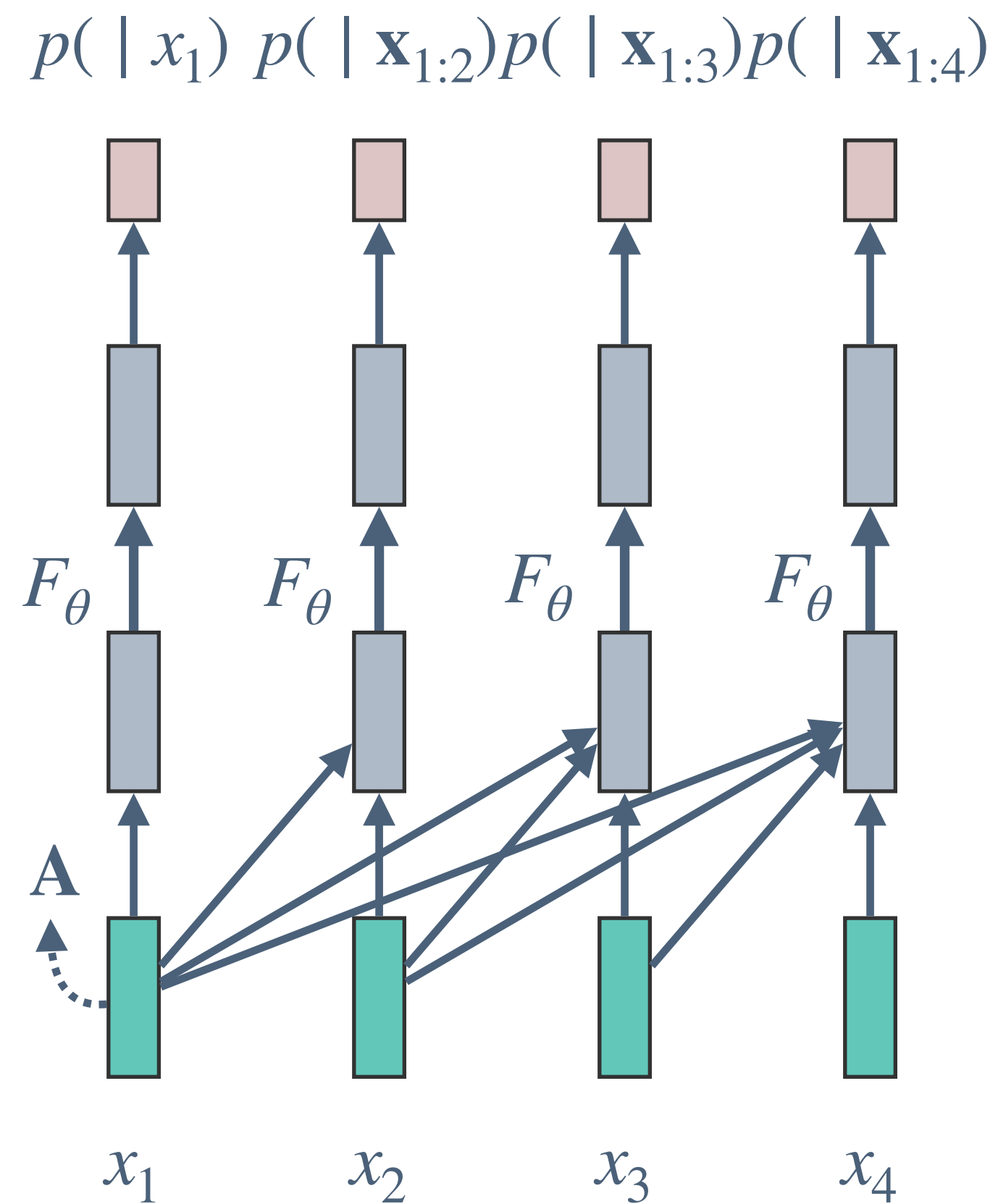


$$\mathbf{T}_{\text{out}}^{(i)} = \text{attn}^{(i)}(\mathbf{T}_{\text{in}}), \text{ para } i \in \{1, 2, \dots, k\}$$

$$\bar{\mathbf{T}}_{\text{out}} = \begin{bmatrix} \mathbf{T}_{\text{out}}^{(1)}[0, :] & \dots & \mathbf{T}_{\text{out}}^{(k)}[0, :] \\ \vdots & \vdots & \vdots \\ \mathbf{T}_{\text{out}}^{(1)}[N-1, :] & \dots & \mathbf{T}_{\text{out}}^{(k)}[N-1, :] \end{bmatrix}_{N \times kv}$$

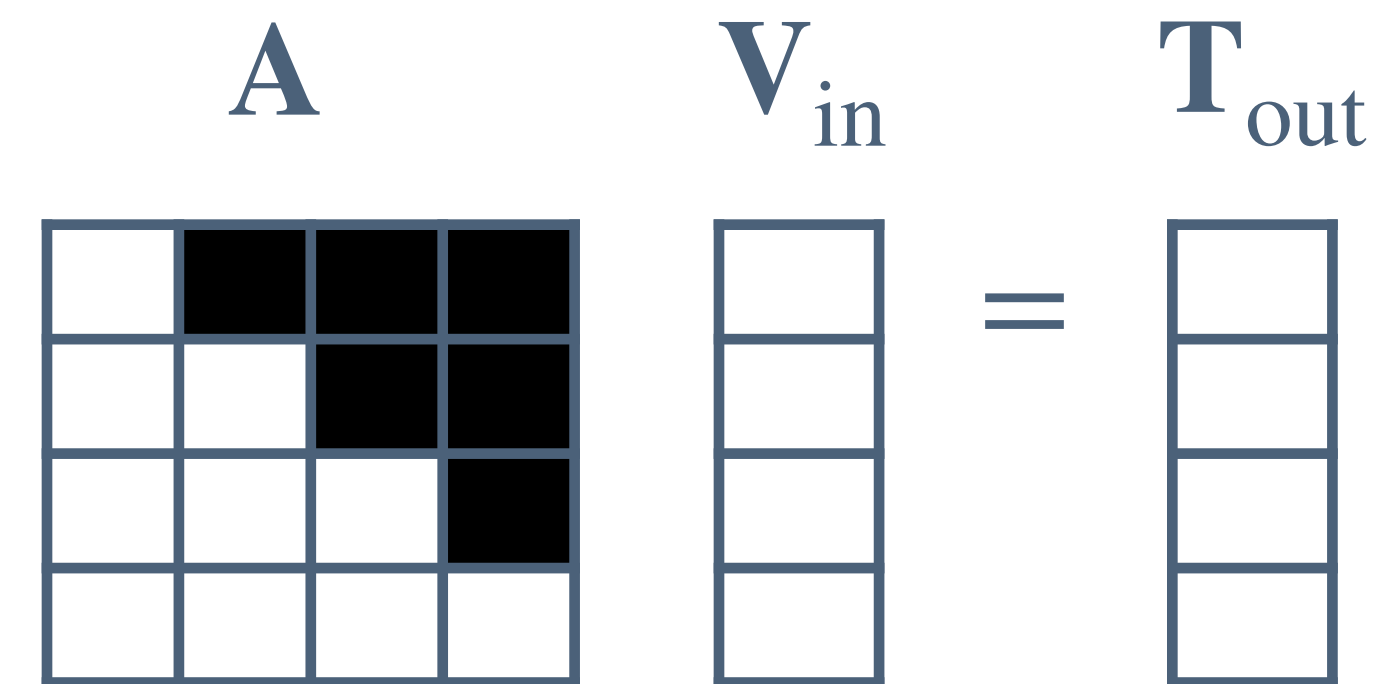
$$\mathbf{T}_{\text{out}} = \bar{\mathbf{T}}_{\text{out}} \mathbf{W}_{\text{MSA}} \quad \mathbf{W}_{\text{MSA}} \in \mathbb{R}^{kv \times d}$$

Transformer Decoder (Causal)



$$\text{Mask}(i, j) = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{Q}_{\text{in}} \mathbf{K}_{\text{in}}^\top}{\sqrt{m}} + \text{Mask} \right)$$



GPT - Generative Pre-Trained Transformer

- Treino **auto-supervisionado** em bases grandes de texto puro (sem anotações).
- Transferência para diferentes tarefas (**Fine-tuning**): Classificação, Implicação, Similaridade, Responder-Questões.

Improving Language Understanding by Generative Pre-Training

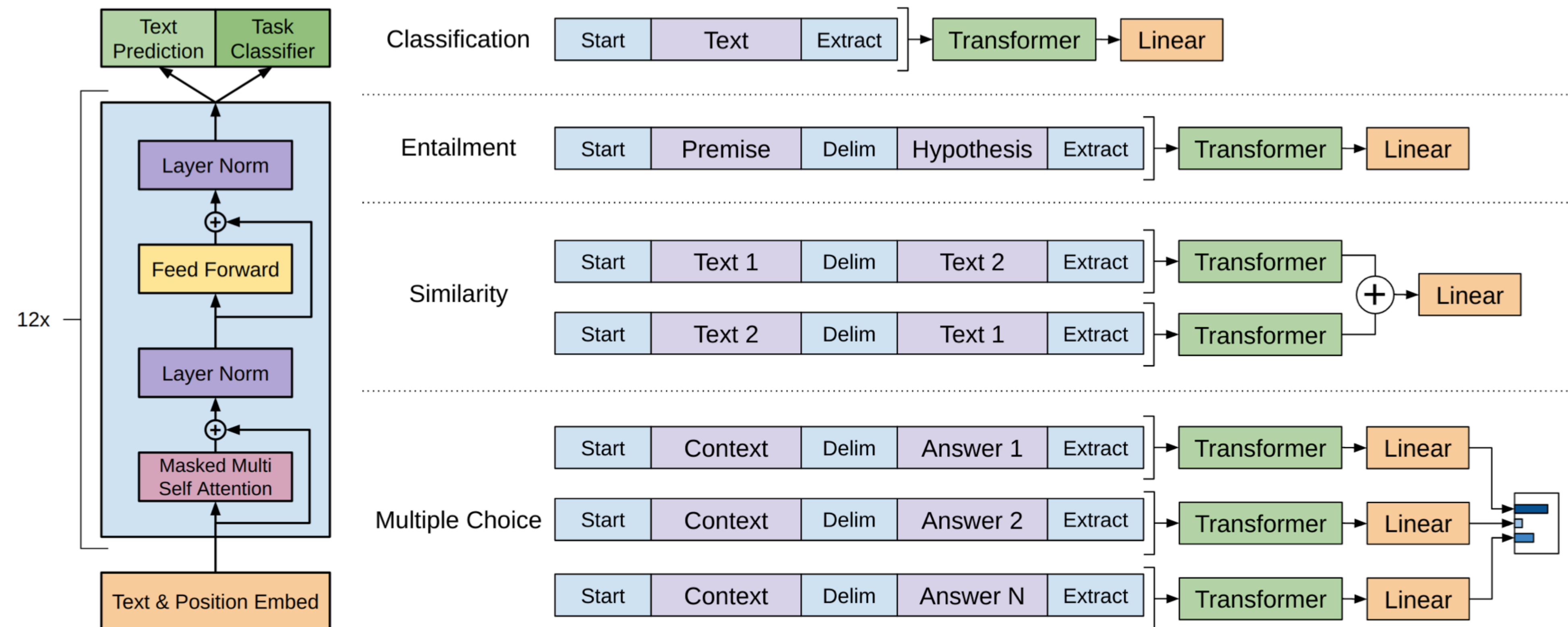
Alec Radford
OpenAI
alec@openai.com

Karthik Narasimhan
OpenAI
karthikn@openai.com

Tim Salimans
OpenAI
tim@openai.com

Ilya Sutskever
OpenAI
ilyasu@openai.com

GPT - Generative Pre-Trained Transformer

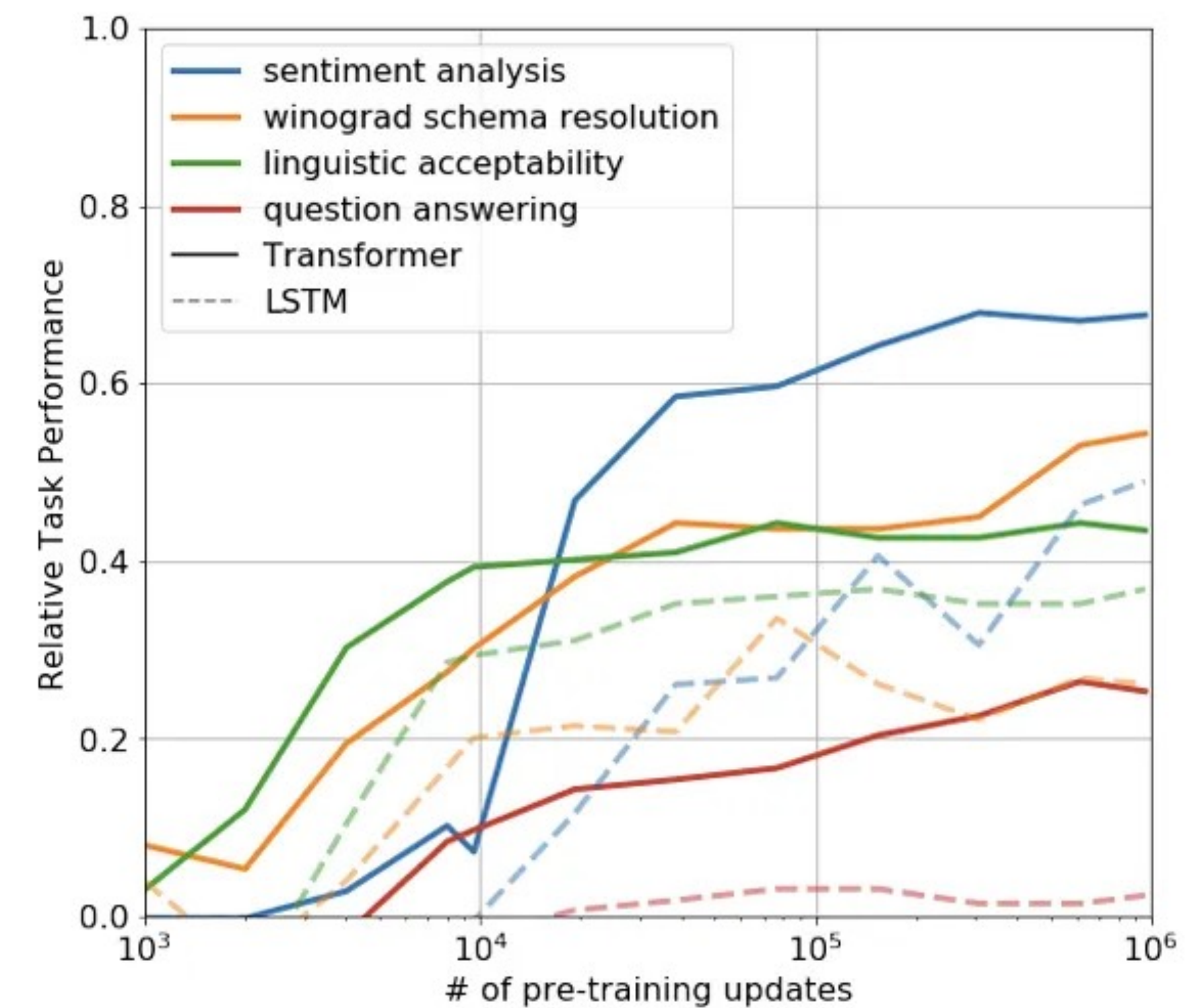


GPT - Generative Pre-Trained Transformer

- **Dados:** BooksCorpus (~7k livros/~5GB).
- **Arquitetura:** Transformer Decoder com 12 camadas (117M param.).
- **Pré-Treino:** Prever a próxima palavra - auto-supervisão
- **Fine-Tuning:** Modificações simples na entrada no modelo permite fine-tuning em diferentes tarefas.
- **Resultados:** O modelo com fine-tuning vence modelos especializados em 9 de 12 benchmarks.

GPT - Generative Pre-Trained Transformer

- Quando Fine-Tuning é realmente necessário?
- Pré-Treino permite LMs aprender diferentes tarefas sem supervisão.



Performance Zero-Shot

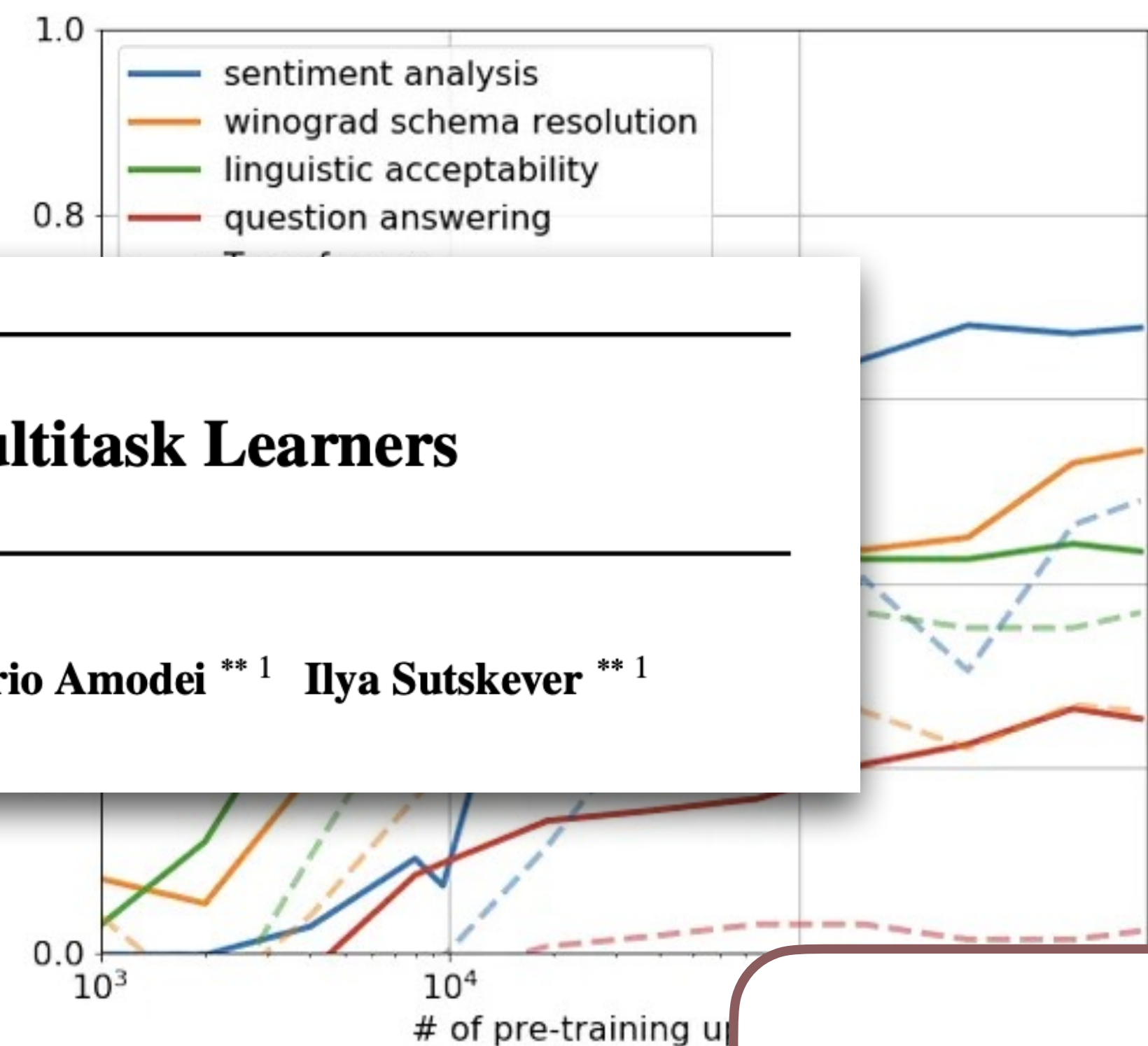
GPT - Generative Pre-Trained Transformer

- Quando Fine-Tuning é realmente necessário?
- Pré-Treino permite LMs aprender diferentes tarefas sem supervisão

Language Models are Unsupervised Multitask Learners

Alec Radford ^{*1} Jeffrey Wu ^{*1} Rewon Child ¹ David Luan ¹ Dario Amodei ^{**1} Ilya Sutskever ^{**1}

GPT-2 Paper



Performance Ze

Data -> 40GB
Param. -> 1.5B

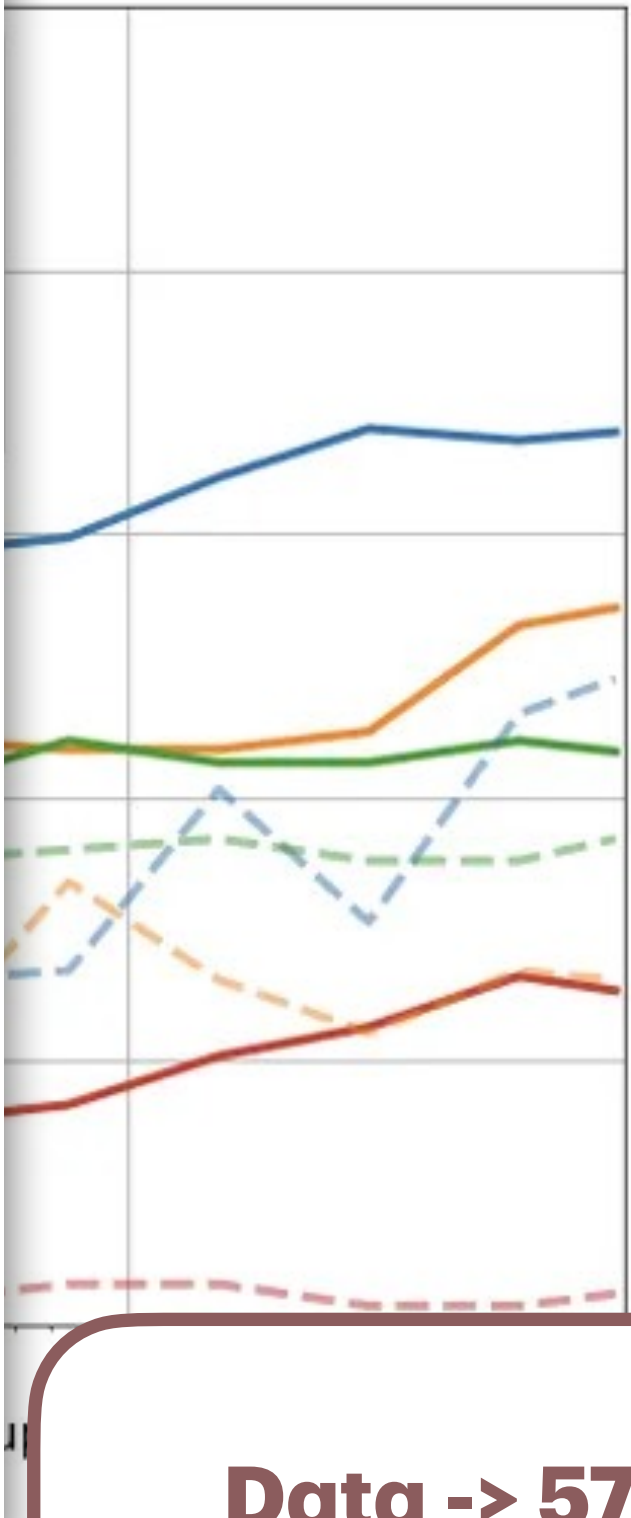
GPT - Generative Pre-Trained Transformer

- Quando Fine-Tune
- Pré-Treino perm
- sem supervisão

Language Models are Few-Shot Learners				
Tom B. Brown*	Benjamin Mann*	Nick Ryder*	Melanie Subbiah*	
Jared Kaplan†	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam	Girish Sastry
Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss	Gretchen Krueger	Tom Henighan
Rewon Child	Aditya Ramesh	Daniel M. Ziegler	Jeffrey Wu	Clemens Winter
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin	Scott Gray
Benjamin Chess		Jack Clark	Christopher Berner	
Sam McCandlish	Alec Radford	Ilya Sutskever	Dario Amodei	
OpenAI				

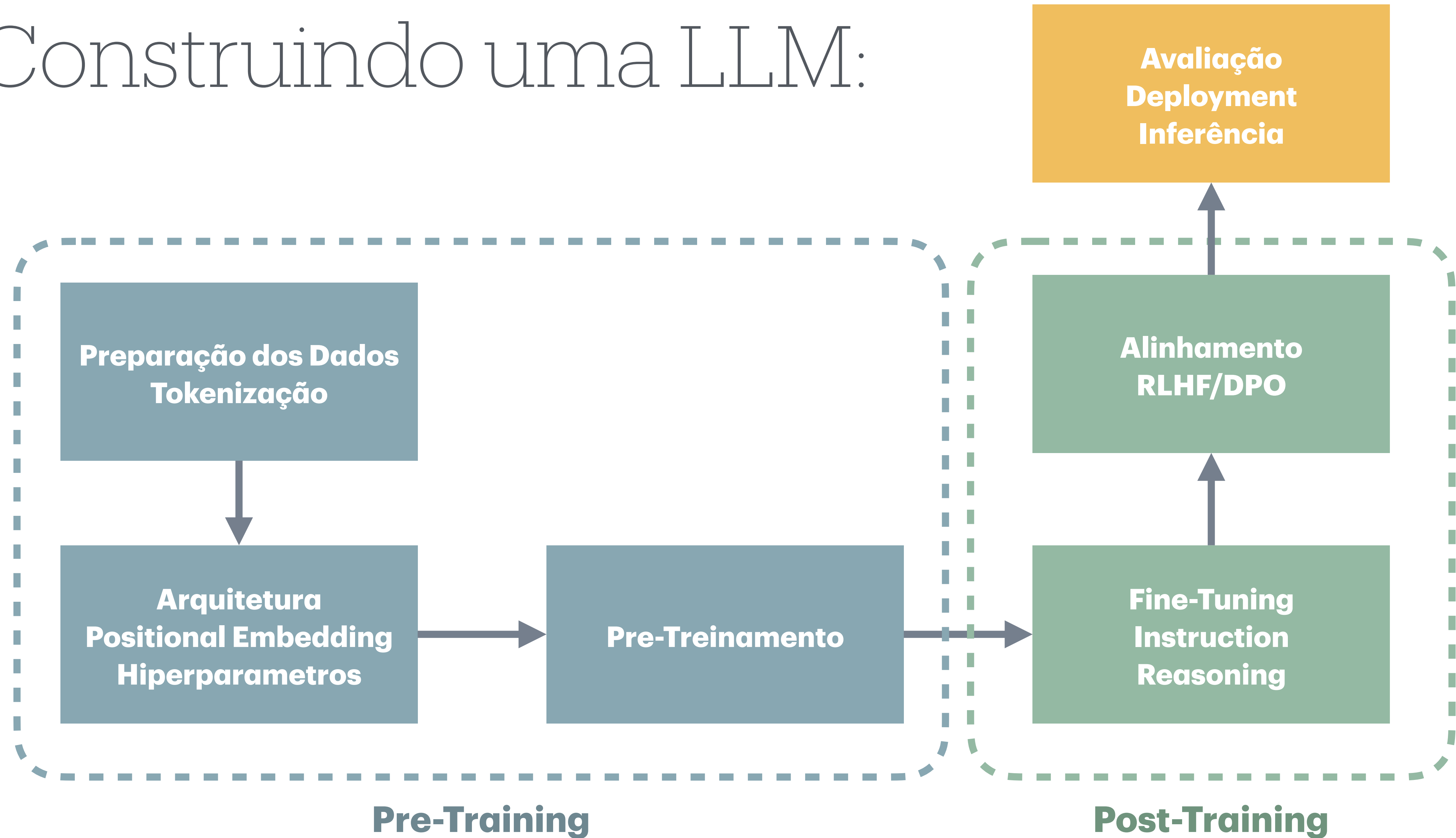
GPT-3 Paper

Improving Language Understanding by Generative Pre-Training



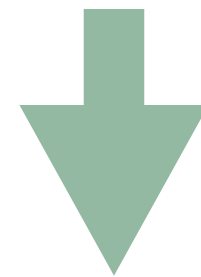
Data -> 570GB
Param. -> 175B

Construindo uma LLM:

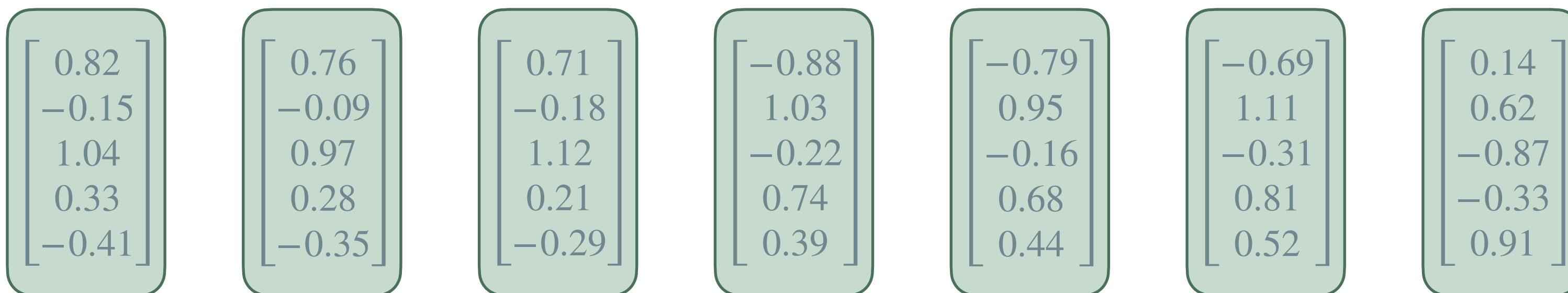


Tokenização

Essa manhã eu levei meu cachorro para



Essa manhã eu levei meu cachorro para



Tokenização

Palavras:

- Ex: jogo, joga, jogar, jogam, jogou, joga -> 6 tokens
- $|\mathcal{V}| \sim 50k$ para uma língua
- $|\mathcal{V}| > 100M$ para Web
- [UNK] -> comum

Caracteres:

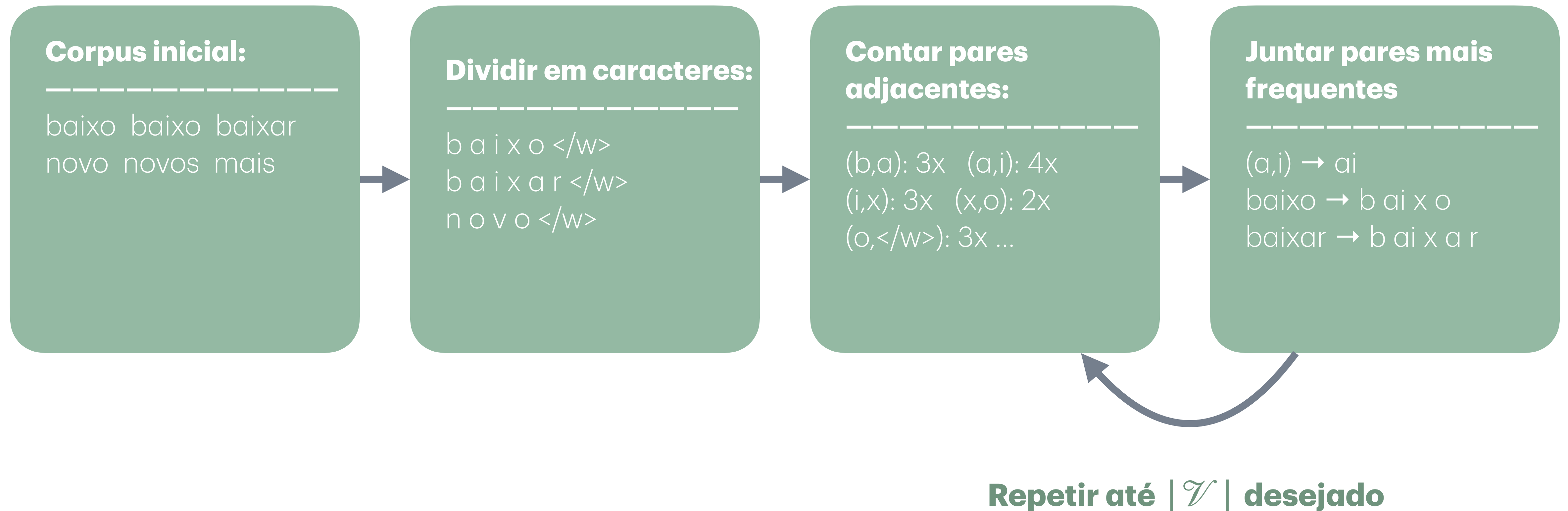
- Ex: Jogador -> 7 tokens
- $|\mathcal{V}| \sim 100-300$
- Sentenças são representadas por **sequências muito longas**

Subpalavras:

- Ex: jogador -> "jog"+"ador"
- Vocabulário compacto
- Sentenças com tamanho razoável
- [UNK] -> raro ou inexistente

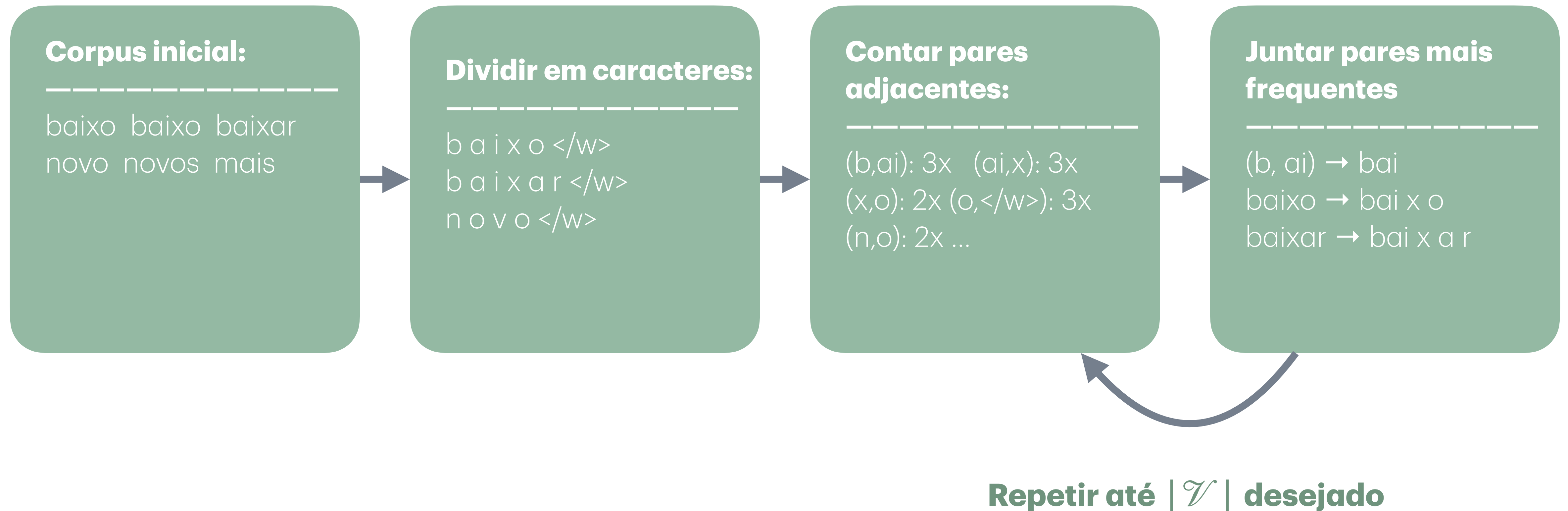
Tokenização

Algoritmo Byte-Pair Encoding (BPE)



Tokenização

Algoritmo Byte-Pair Encoding (BPE)



Tokenização

Algoritmo Byte-Pair Encoding (BPE)

Exemplos:

- jogadores -> jog + ador + es
- computador -> comput + ador
- brincando -> brinc + ando
- falando -> fal + ando
- tokenização -> token + ização

Tokenização

Algoritmo Byte-Pair Encoding (BPE)

Operações por byte:

- 1 byte = 8 bits — 256 valores possíveis
- UTF-8 codifica qualquer caractere do como sequência de 1–4 bytes:
 - A → 1 byte (65)
 - é → 2 bytes (195, 169)
 - 中 → 3 bytes (228, 184, 173)
 - 😊 → 4 bytes (240, 159, 152, 138)

Tokenização

Algoritmo Byte-Pair Encoding (BPE)

- **Treino :** Aprende regras para juntar bytes (centenas de GB pode levar 1 dia)
- **Tokenização:** Quebras sentença em bytes e aplicar as regras até obter os tokens (milhões de tokens por segundo em uma CPU)
- O maior custo envolvido com a tokenização é na matrix de embedding associada. Para cada token associamos um vetor. Então precisamos manter uma tabela de $|\mathcal{V}| \times d$ que pode ser grande ($\sim 50k \times 4096 \approx 800$ MB).

Positional Embeddings

Embedding sinusoidal:

$$E(x, i) = v_x + PE_i$$

$$PE_{i,2j} = \sin\left(\frac{i}{n^{2j/d}}\right) \quad PE_{i,2j+1} = \cos\left(\frac{i}{n^{2j/d}}\right)$$

Embedding absoluto:

$$E(x, i) = v_x + u_i$$

Embedding relativo:

(no cálculo da atenção)

$$e_{i,j} = \frac{x_i W^Q \left(x_j W^K + a_{i,j}^K \right)^T}{\sqrt{d_z}}$$

RoPE: Rotatory Position Embeddings

Gostaríamos de ter uma position embedding tal que a atenção entre os tokens seja dada em **função de suas posições relativas**,

$$\langle f(q, i), f(k, j) \rangle = g(q, k, i - j)$$

o que importa para comparar a importância de k para q deveria ser a posição relativa entre os tokens e não o número da posição em si.

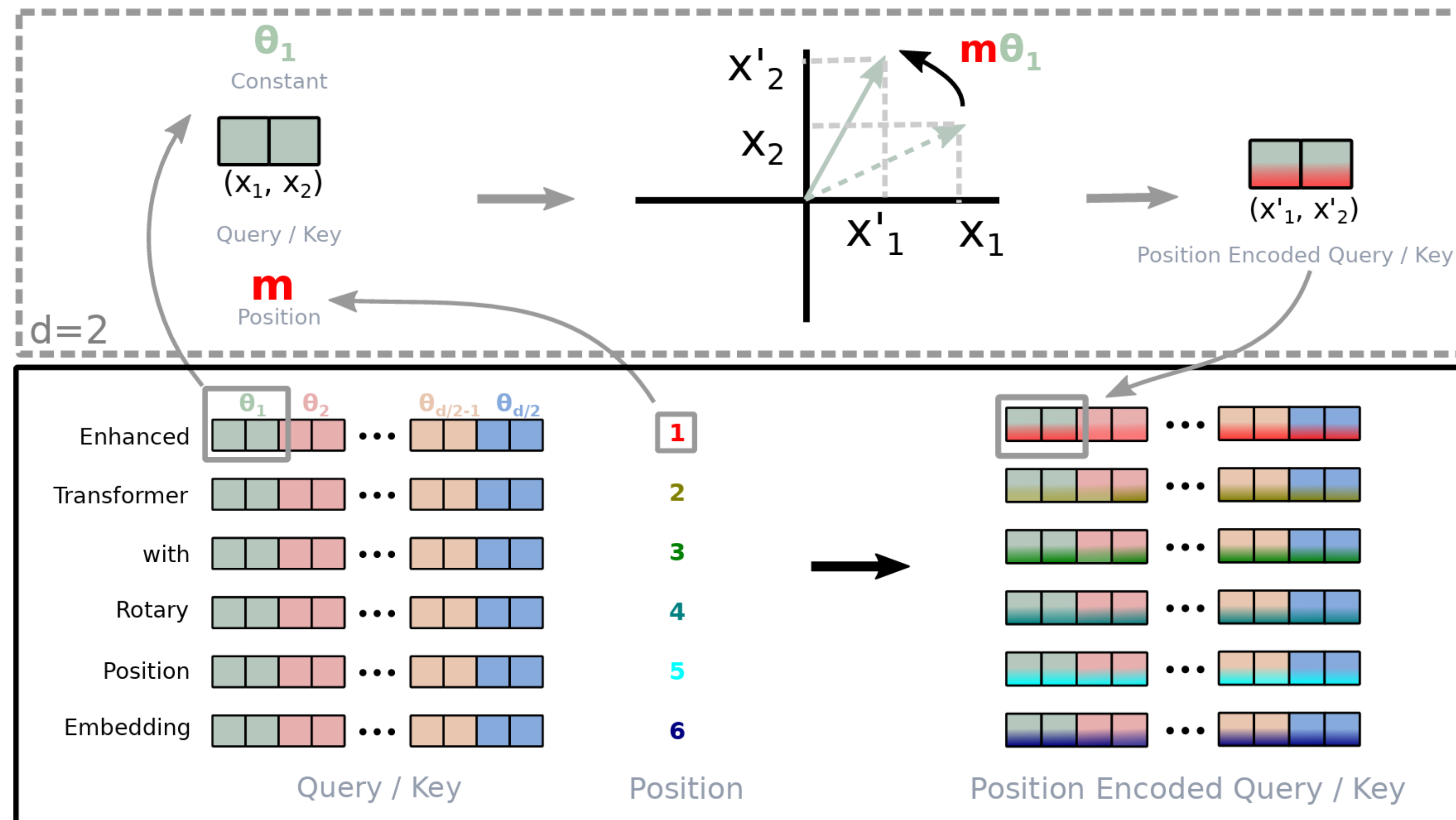
RoPE: Rotatory Position Embeddings

Em RoPE codificamos a posição usando rotações. Dado que $\langle x, y \rangle = \|x\| \|y\| \cos(\theta_{xy})$ é invariante por rotações, um mesmo shift na posição de dois vetores mantém a posição relativa.



RoPE: Rotatory Position Embeddings

Como fazer isso para vetores em alta dimensão?



RoPE: Rotatory Position Embeddings

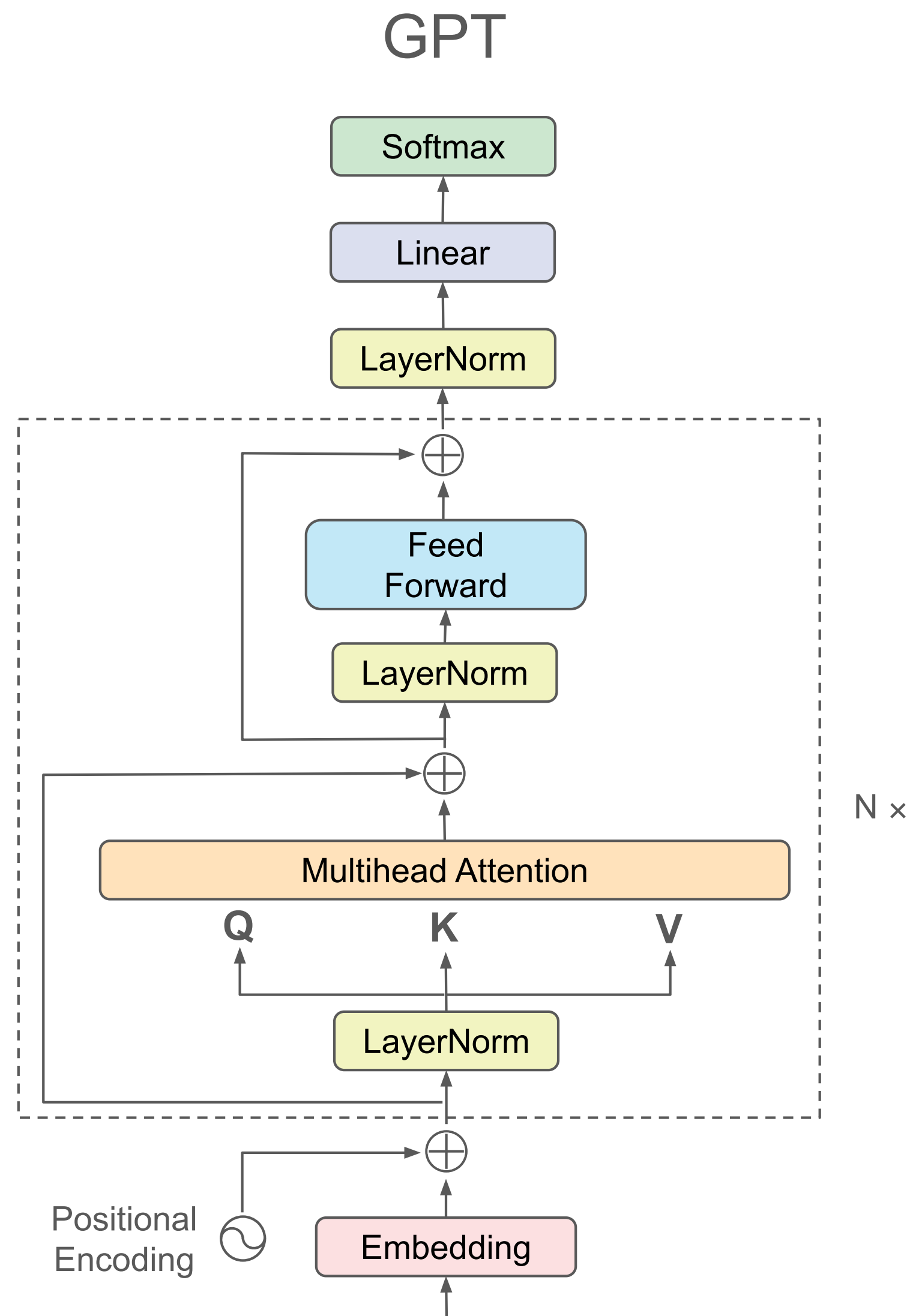
As operações podem ser escritas de forma matricial como

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

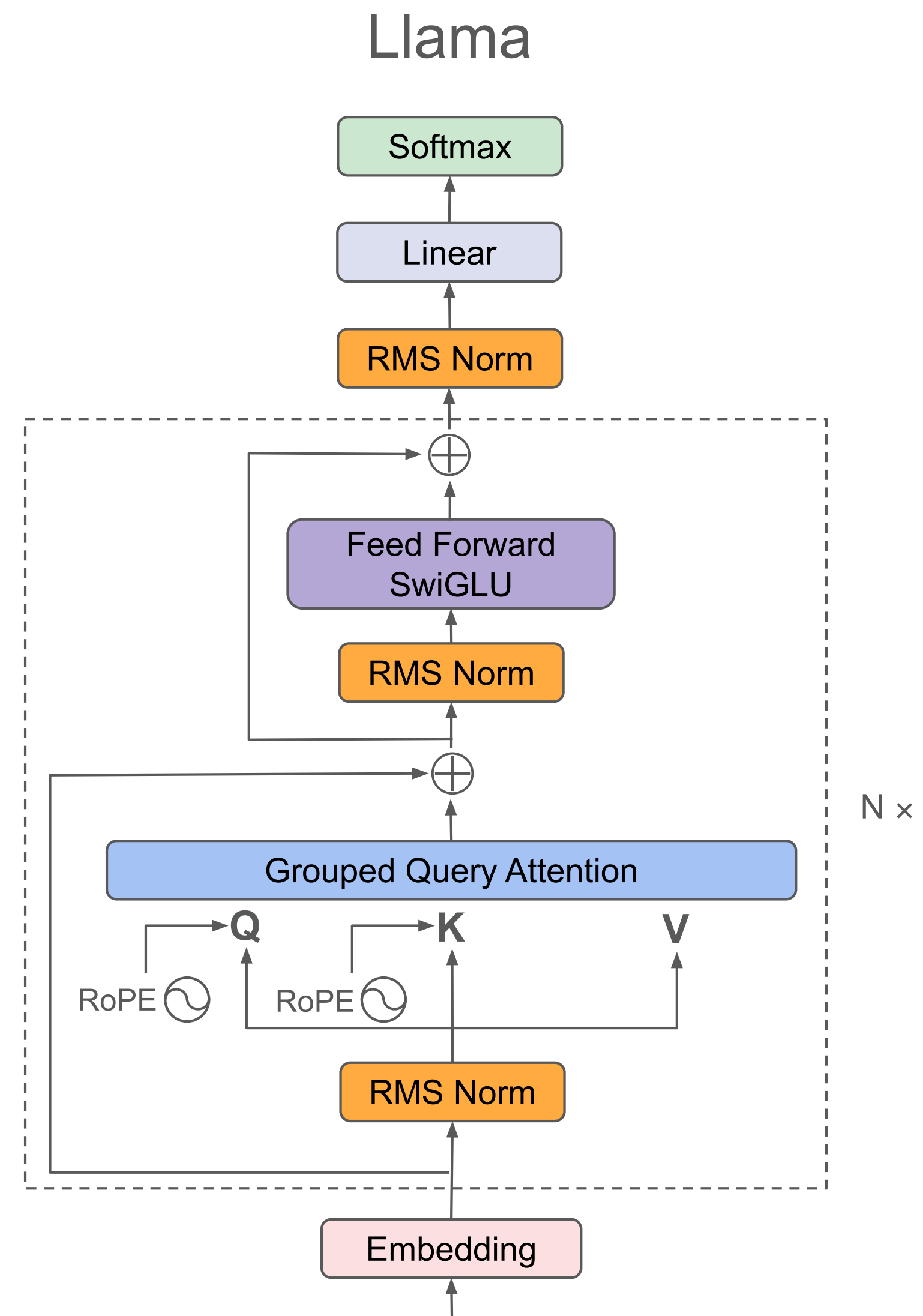
$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

$$\theta_i = 10000^{-2i/d}$$

Arquiteturas



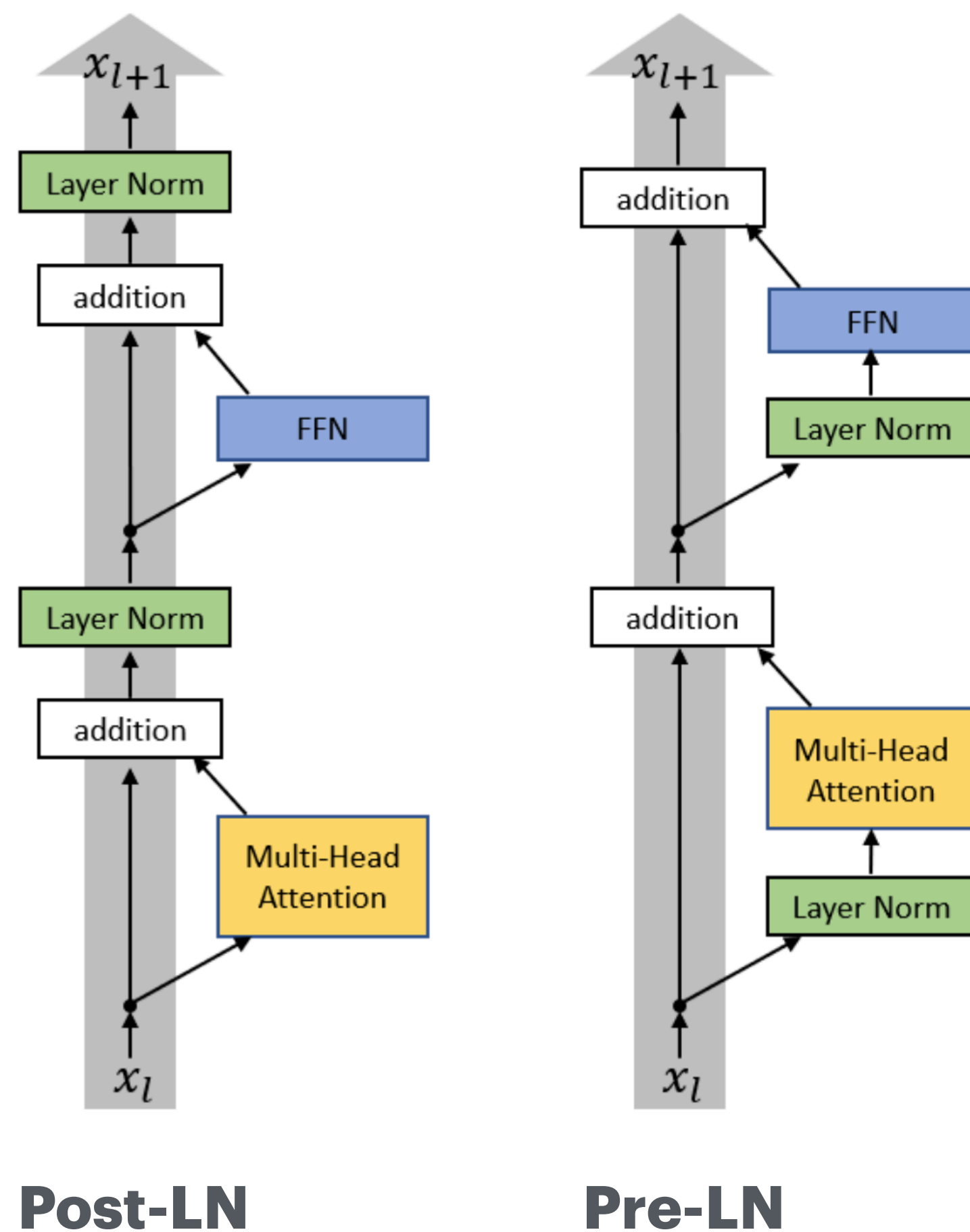
vs



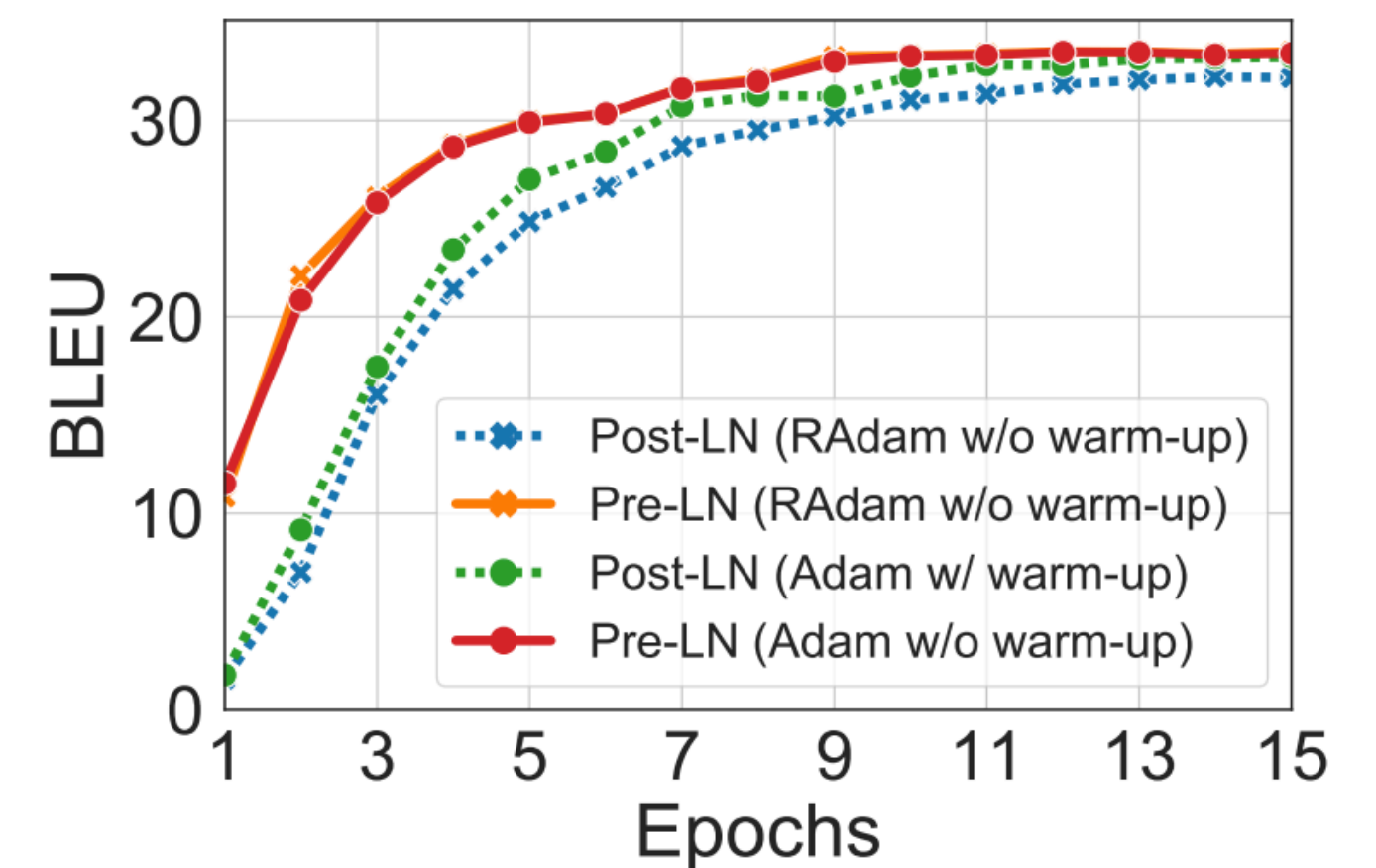
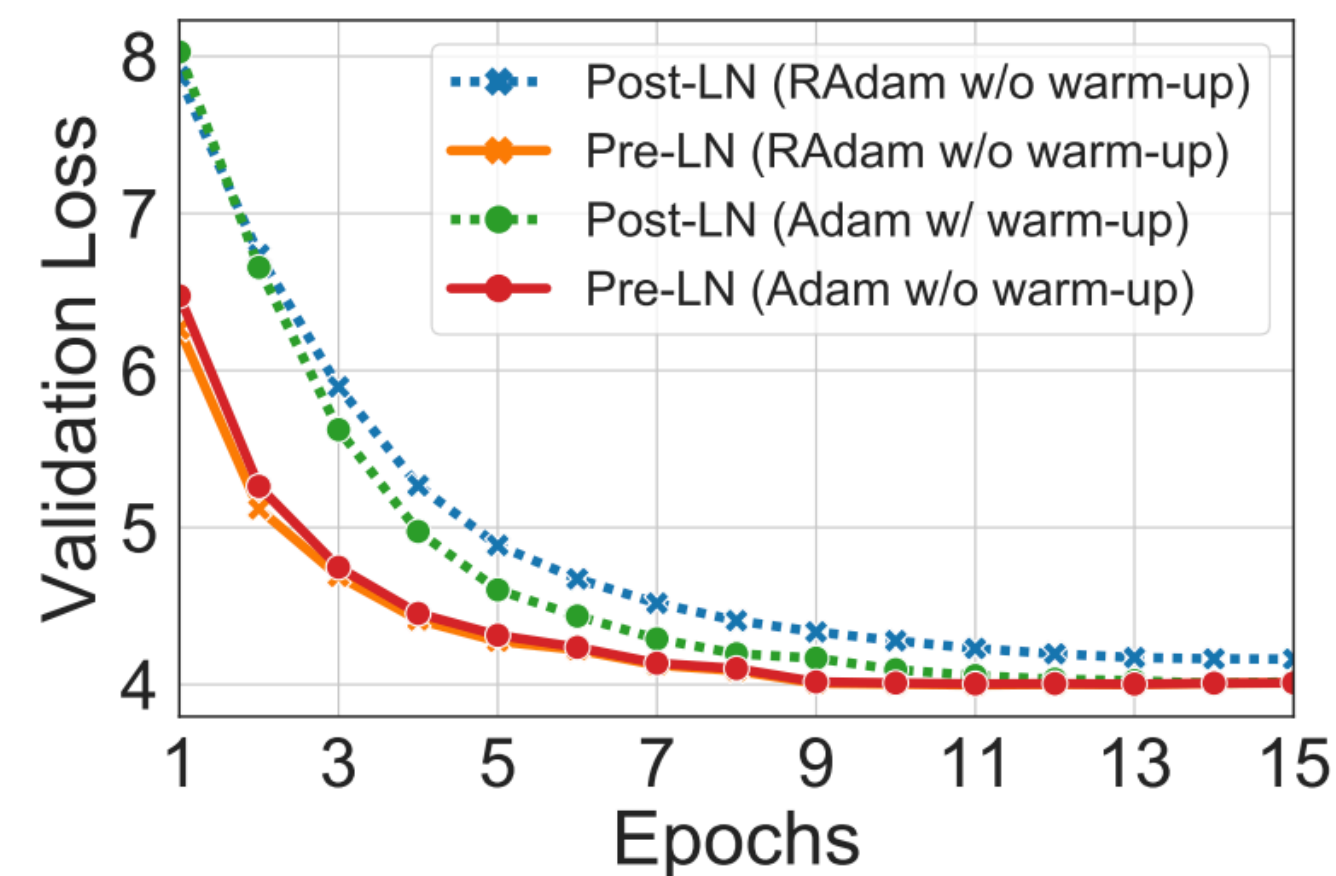
Arquiteturas

- Estratégias de Normalização
- Funções de Ativação
- Positional Embedding
- Hiperparâmetros - dimensões, tamanhos, proporções
- Truques de Estabilidade

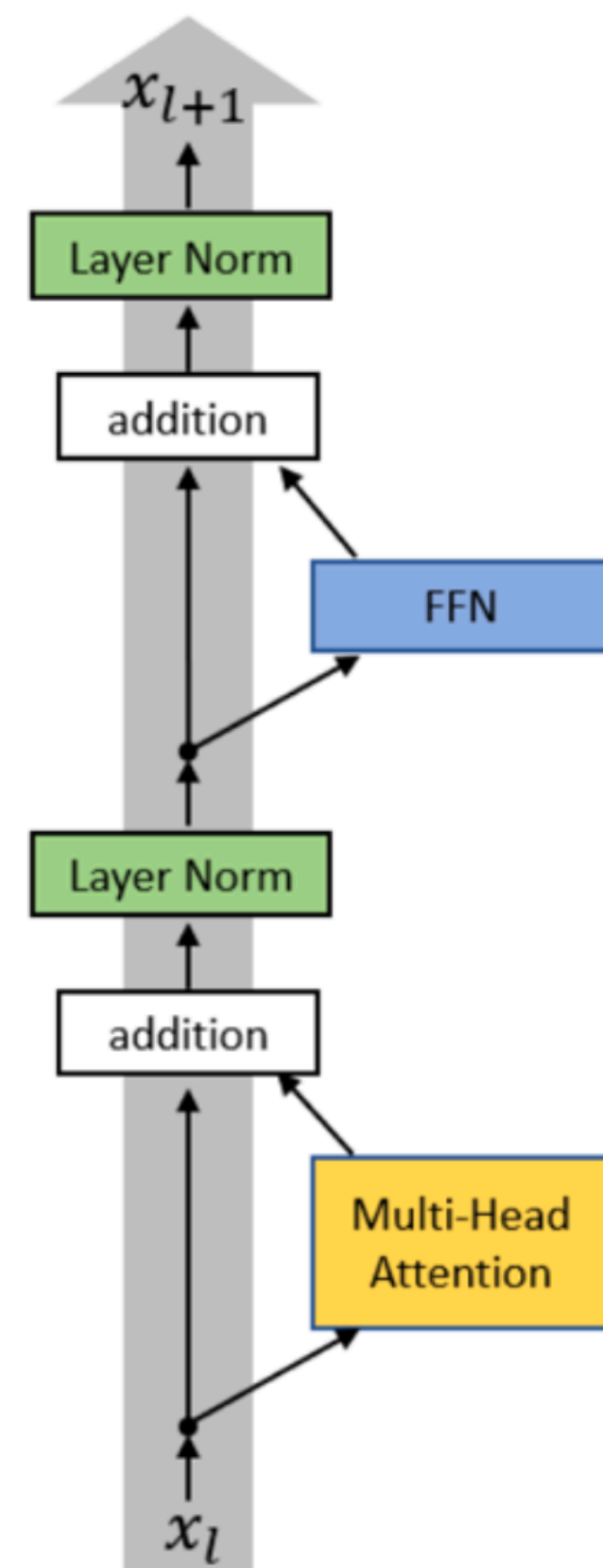
Pre-vs-Post Norm



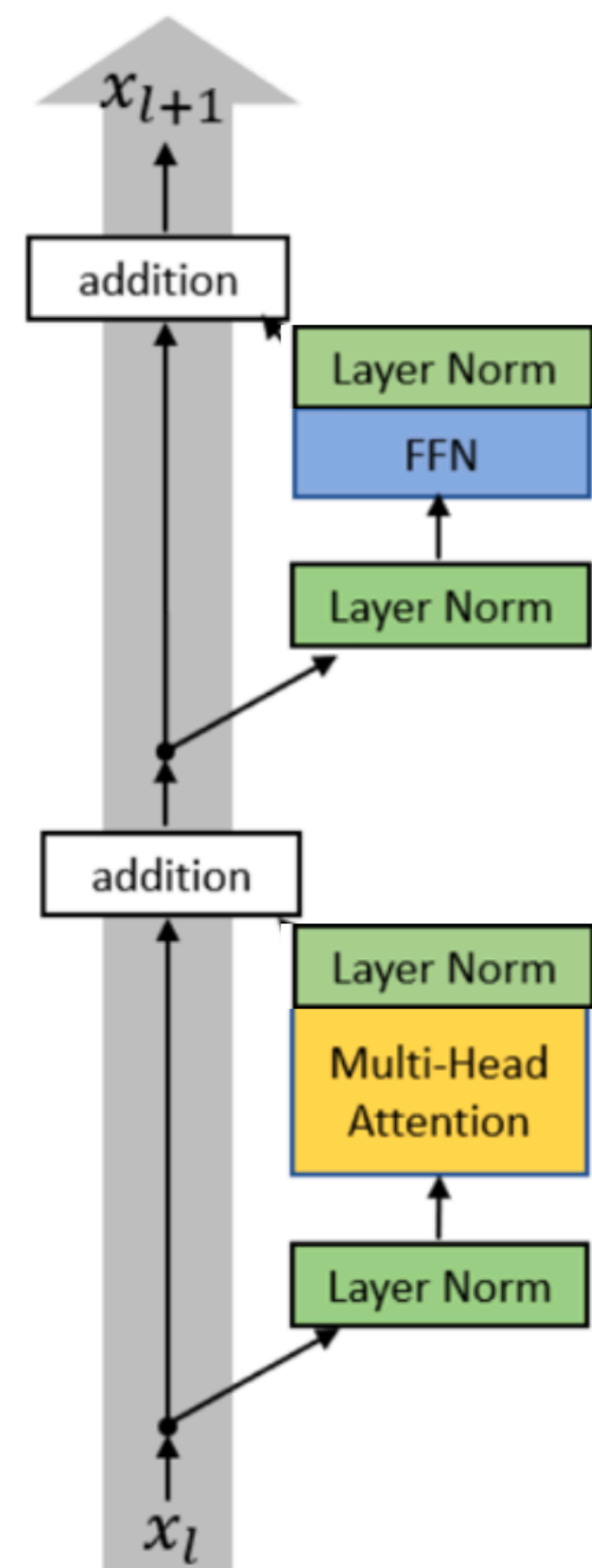
- Pre-LN mantém melhor a magnitude do gradiente no sinal residual
- Em geral, LLMs usam um esquema Pre-LN



Pre-vs-Post Norm



Post-LN



Pre-LN

- Alguns modelos recentes como Grok e Gemma 2 adicionam um Post-LN a mais, porém fora do caminho residual — “double” norm.
- Parece ajudar com estabilidade

LayerNorm vs RMSNorm

- **LayerNorm** normaliza a representação de cada token usando a média e variância do próprio vetor. Além de aplicar uma escala e bias aprendidos

$$x_{\text{out}}[k] = \frac{x_{\text{in}}[k] - \mathbb{E}[x_{\text{in}}]}{\sqrt{\text{Var}(x_{\text{in}})}} * \gamma + \beta$$

- **RMSNorm** normaliza sem a centralização ou escala da variância — ganho em eficiência

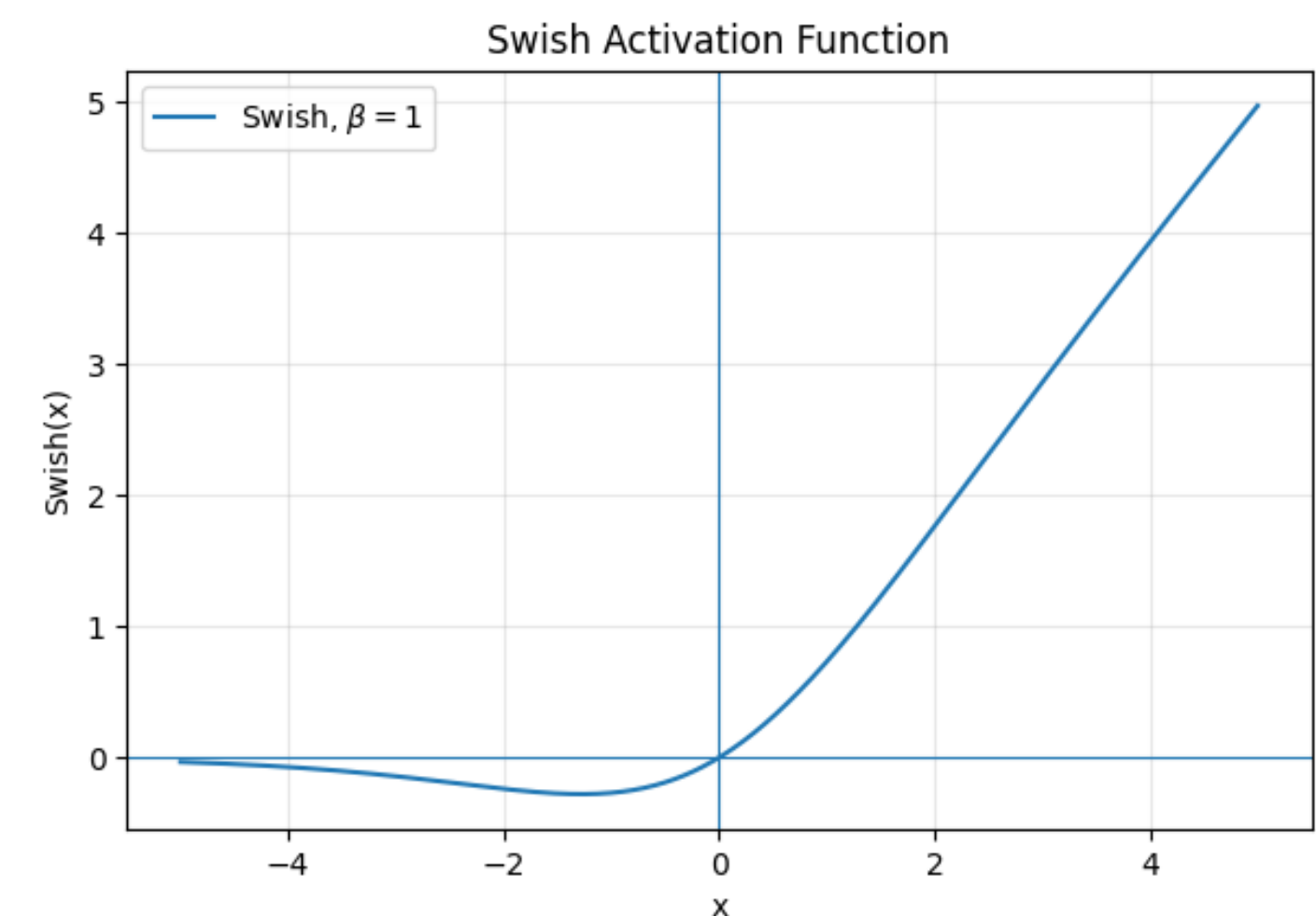
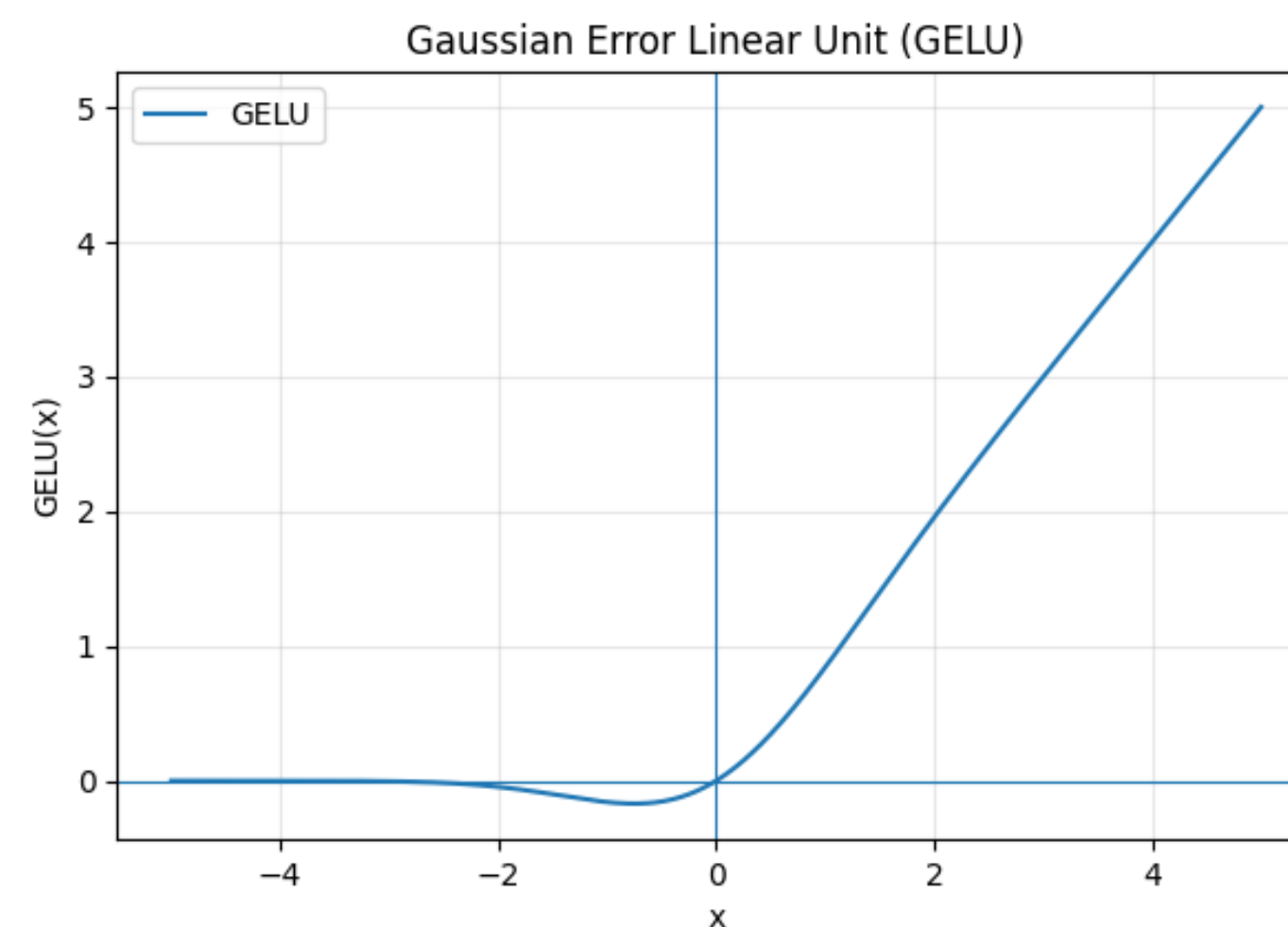
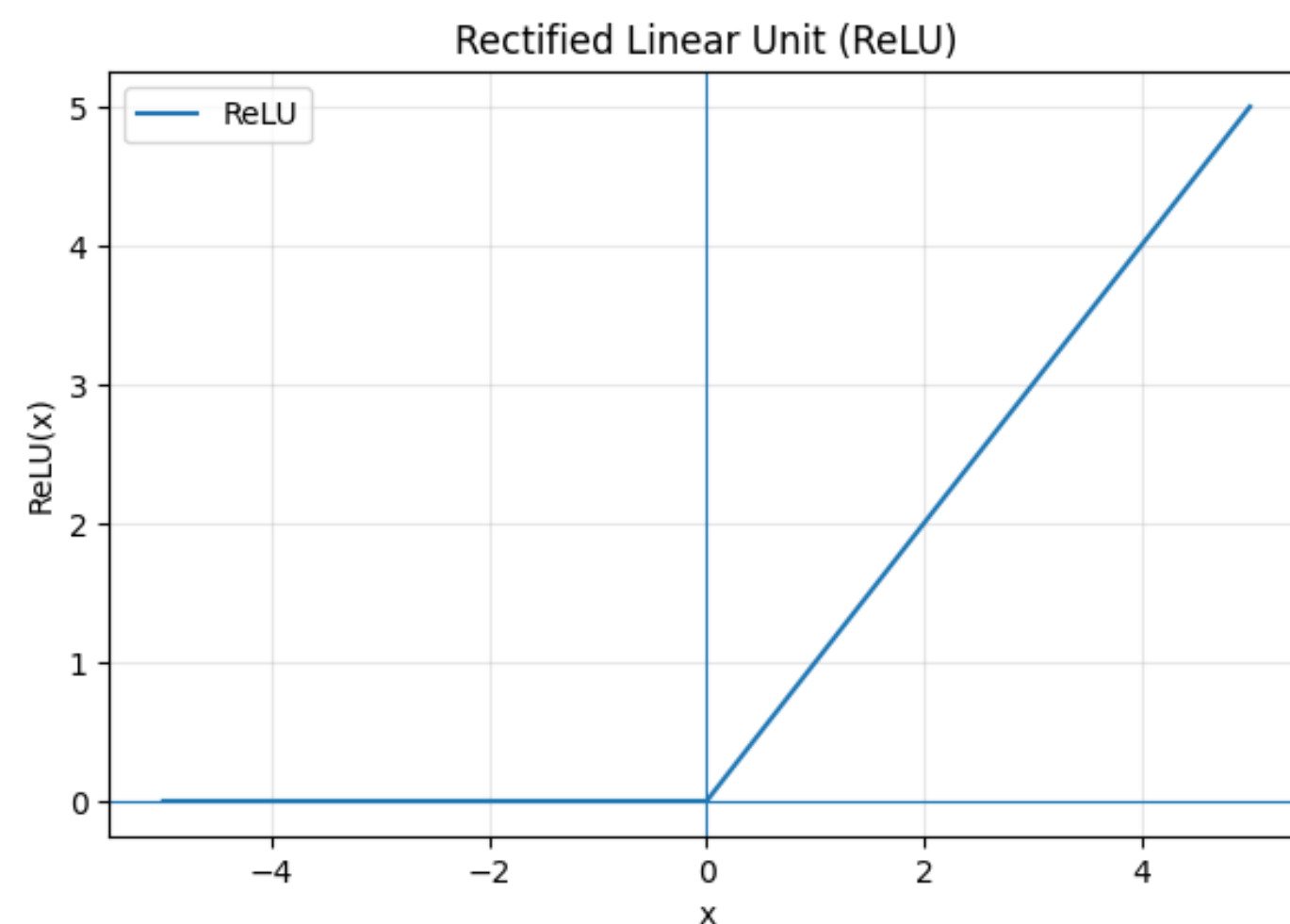
$$x_{\text{out}} = \frac{x_{\text{in}}}{\sqrt{\|x_{\text{in}}\|^2 + \epsilon}} * \gamma$$

Arquitetura Feed-Forward

A maioria dos transformers modernos **não possuem termo de viés**

$$\text{FFN}(x) = W_2 \max(0, W_1 x + b_1) + b_2 \quad \longrightarrow \quad \text{FFN}(x) = W_2 \sigma(W_1 x)$$

Funções de ativação: ReLU, GeLU, Swish, ELU, GLU, GeGLU, ReGLU, ...



Gated Linear Units (GLU)

Ativações com mecanismo de porta

Semelhante como em LSTMs, consideramos uma transformação linear que controla qual parte da informação é transmitida ($\otimes$ multiplica coordenada a coordenada)

$$\sigma(W_1x) \longrightarrow \sigma(W_1x) \otimes (Vx)$$

ReGLU:

$$\text{FFN}_{\text{ReGLU}}(x) = W_2 \max(0, W_1x) \otimes (Vx)$$

GeGLU:

$$\text{FFN}_{\text{GeGLU}}(x) = W_2 (\text{GeGLU}(W_1x) \otimes (Vx))$$

SwiGLU:

$$\text{FFN}_{\text{GeGLU}}(x) = W_2 (\text{Swish}(W_1x) \otimes (Vx))$$

Gated Linear Units (GLU)

Ativações com mecanismo de porta

	Score Average	CoLA MCC	SST-2 Acc
FFN _{ReLU}	83.80	51.32	94.04
FFN _{GELU}	83.86	53.48	94.04
FFN _{Swish}	83.60	49.79	93.69
FFN _{GLU}	84.20	49.16	94.27
FFN _{GEGLU}	84.12	53.65	93.92
FFN _{Bilinear}	83.79	51.02	94.38
FFN _{SwiGLU}	84.36	51.59	93.92
FFN _{ReGLU}	84.67	56.16	94.38
[Raffel et al., 2019]	83.28	53.84	92.68
ibid. stddev.	0.235	1.111	0.569

Hiperparâmetros

- O tamanho padrão para a camada escondida das redes Feed-forward é $d_{ff} = 4 d_{\text{model}}$, quando usamos GLU esse número é $d_{ff} = \frac{8}{3} d_{\text{model}}$.
- Para atenção com múltiplas cabeças o padrão é que $d_H = d_{\text{model}}/n_H$, porém algo um pouco maior também pode funcionar bem.

Pre-Treinamento

- **Preparação dos Dados:** deduplicação, segurança, privacidade, contaminação
- **Tokenização**
- **Escolha da Arquitetura**
- **Pré-Treino — Prever a próxima palavra**

$$\hat{\theta} = \arg \max_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \mathcal{L}_{\theta}(\mathbf{x}) \quad \mathcal{L}_{\theta}(\mathbf{x}) = \sum_{i=1}^k \log p_{\theta} \left(x_i \mid \mathbf{x}_{1:i-1} \right)$$

- **Otimização:** AdamW, Learning-rate scheduler, Dropout (?), Paralelismo, ...

Avaliação

- **Perplexity** — média geométrica da verosimilhança — escala independente do tamanho da sequência

$$\text{PPL}_{\theta}(\mathbf{x}_{1:n}) = \left(\prod_{k=1}^n p \left(x_k \mid \mathbf{x}_{1:k-1} \right) \right)^{-\frac{1}{n}}$$

- **Zero-shot/Few-shot benchmarks**